东北大学 国家示范性软件学院
SOFTWARE COLLEGE OF NORTHEASTERN UNIVERSITY

Python for Data Analysis
—— PART II Improved

1

雨课堂 Rain Classroom

Preliminaries

- Suppose you want to calculate the dot product of two vectors with larger dimensions, that is, a*b,

```
import numpy as np
import time
a = np.random.rand(1000000)
b = np.random.rand(1000000)
c = 0
tic = time.time()
for i in range(1000000):
    c+=a[i]*b[i]
toc = time.time()
print(f"矩阵内积为：{c}")
print(f"使用循环的运行时间为：{1000*(toc-tic)}ms")
```

Output Result:

矩阵内积为：249841.56804663077

使用循环的运行时间为：609.9998950958252ms

$$\begin{bmatrix} a_1 \\ a_2 \\ a_3 \\ \cdots \\ a_n \end{bmatrix} * \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ \cdots \\ b_n \end{bmatrix}$$

雨课堂 Rain Classroom

Preliminaries

- Suppose you want to calculate the dot product of two vectors with larger dimensions, that is, a*b,

```
import numpy as np
import time
a = np.random.rand(1000000)
b = np.random.rand(1000000)
c = 0
tic = time.time()
c = np.dot(a,b)
toc = time.time()
print(f"矩阵内积为：{c}")
print(f"使用循环的运行时间为：{1000*(toc-tic)}ms")
Output Result:
矩阵内积为：249831.8201744269
使用循环的运行时间为：2.0062923431396484ms
```

$$\begin{bmatrix} a_1 \\ a_2 \\ a_3 \\ \cdots \\ a_n \end{bmatrix} * \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ \cdots \\ b_n \end{bmatrix}$$

雨课堂 Rain Classroom

Preliminaries

- Suppose a vector v with a large dimension is known, and you want to calculate u

```
import numpy as np
import math, time
v = np.random.rand(1000000)
u = np.zeros((1000000,1))
tic = time.time()
for i in range(1000000):
    u[i]=math.exp(v[i])
toc = time.time()
print(f"使用循环的运行时间为：{1000*(toc-tic)}ms")
tic = time.time()
u = np.exp(v)
toc = time.time()
print(f"使用向量化的运行时间为：{1000*(toc-tic)}ms")
```

Output Result:

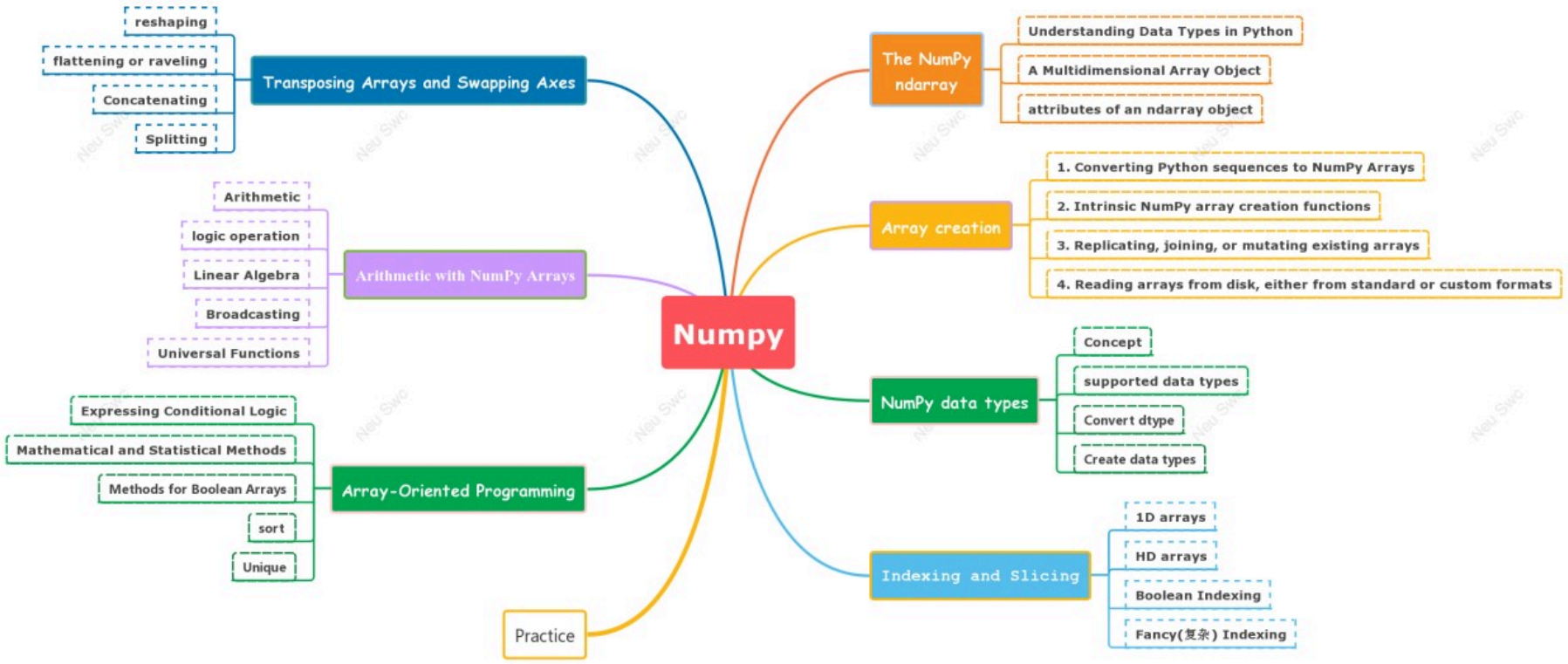
使用循环的运行时间为：979.5792102813721ms

使用向量化的运行时间为：12.633800506591797ms

$$\mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ \cdots \\ v_n \end{bmatrix} \Rightarrow \mathbf{u} = \begin{bmatrix} e^{v_1} \\ e^{v_2} \\ e^{v_3} \\ \cdots \\ e^{v_n} \end{bmatrix}$$

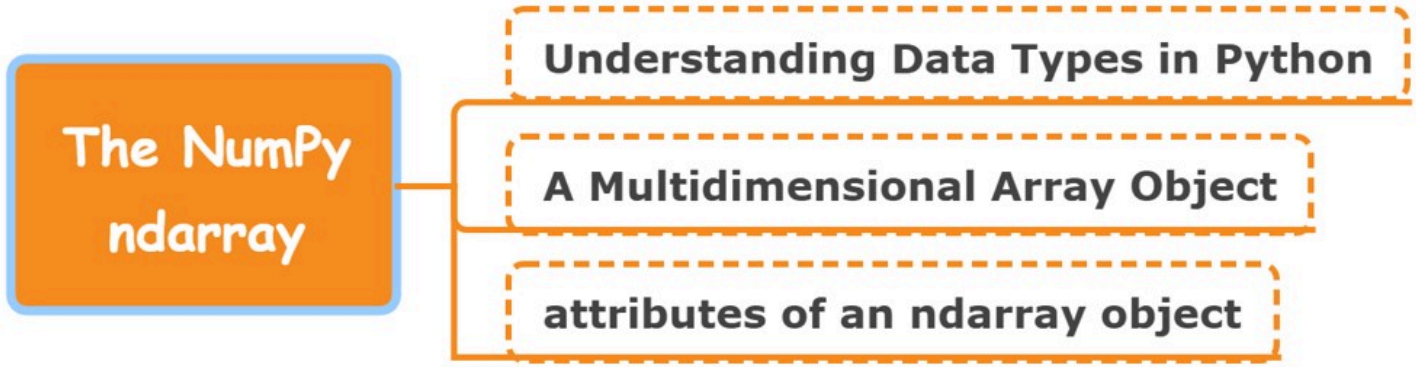
雨课堂 Rain Classroom

Lecture2: NumPy Basics



5

Lecture2: NumPy Basics



Understanding Data Types in Python

1. Integer in Python is not just an integer

- The implementation of standard Python code is written in C.
- This means that every Python object is just a cleverly disguised(伪装) C structure that contains not only its value, but other information as well.
  - For example, when we define an integer in Python, "**x = 10000**", *x* is not just a ***primitive***(原始) integer.
  - It's actually a pointer to a composite C structure that contains several values.

Understanding Data Types in Python

```
struct _longobject {
        long ob_refcnt;          # 一个帮助Python静默处理内存分配和释放的引用计数
        PyTypeObject *ob_type;   # 变量的类型
        size_t ob_size;          # 指定以下数据成员的大小
        long ob_digit[1];        # 包含我们期望Python变量表示的实际整数值
};
```

- 注意：
  - C语言中，int本质上是一个在内存中位置的标签，它是字节编码的整数值。
  - Python语言中，int是指向包含所有Python对象信息的内存位置的指针，包括包含整数值的字节。
  - Python整数结构中的这些额外信息使得Python能够如此自由和动态地编码。
- 然而，在Python类型中，所有这些附加信息都是以成本为代价的，在组合了许多这些对象的结构中，这一点变得尤为明显。

8

Understanding Data Types in Python

2. List in Python is not just a list

```
>>> L = [True, "2", 3.0, 4]
>>> [type(item) for item in L]
Output Result:
[bool, str, float, int]
```

- Python列表包含指向一个指针块的指针，每个指针都指向一个完整的Python对象，就像我们前面看到的Python整数。
- 如果在所有变量都是相同类型的特殊情况下，大部分信息是冗余的
- 所以在固定类型的数组中存储数据会更有效。
- 在实现级别，数组本质上包含一个指向连续数据块的指针。

Understanding Data Types in Python

3. array in Python

```
>>> import array
>>> L = list(range(10))
>>> A = array.array("i", L)
>>> A
Output Result:
array('i', [0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

这里'i' 表示内容为整数的类型代码

- Python提供了几种不同的选项，用于在高效的固定类型数据缓冲区中存储数据。
- 内置的“array”模块(从Python 3.3开始可用)可用于创建统一类型的密集数组:
- 但是array模块不支持多维，也没有各种运算函数。
- **固定类型的numpy样式的数组缺少这种灵活性，但是对于存储和操作数据来说更有效。**

10

The NumPy ndarray

A Multidimensional Array Object

- ***ndarray***(存储单一数据类型的多维数组), an efficient multidimensional array providing fast array-oriented arithmetic operations and flexible ***broadcasting*** capabilities.
- **Mathematical functions** for fast operations on entire arrays of data **without having to write loops**.
- Tools for reading/writing array data to disk and working with memory-mapped files.
- Linear algebra, random number generation, and Fourier transform(傅里叶变换) capabilities.
- A ***C API*** for connecting NumPy with libraries written in C, C++, or FORTRAN.
- While NumPy by itself does not provide modeling or scientific functionality, having an understanding of NumPy arrays will help you use tools like pandas, much more effectively.

The NumPy ndarray

A Multidimensional Array Object

import numpy as np

- 不要使用 from numpy import *，容易混乱
- numpy中有大量与Python内置函数同名的函数，如min，max等

https://docs.scipy.org/doc/numpy/genindex.html

- One of the reasons NumPy is so important for numerical computations in Python is because it is designed for efficiency on **large arrays of data**. There are a number of reasons for this:
  - NumPy internally stores data in a contiguous连续的 block of memory, independent of other built-in Python objects.
  - NumPy's library of algorithms written in the C language can operate on this memory without any type checking or other overhead开销.
  - NumPy arrays also use much less memory than built-in Python sequences.

12

The NumPy ndarray

A Multidimensional Array Object

- In the first example, all of the elements have been multiplied by 10.
- In the second, the corresponding values in each "cell" in the array have been added to each other.

```
In [12]: import numpy as np

# Generate some random data
In [13]: data = np.random.randn(2, 3)

In [14]: data
Out[14]:
array([[-0.2047,  0.4789, -0.5194],
       [-0.5557,  1.9658,  1.3934]])
```

```
In [15]: data * 10
Out[15]:
array([[ -2.0471,   4.7894,  -5.1944],
       [ -5.5573,  19.6578,  13.9341]])

In [16]: data + data
Out[16]:
array([[-0.4094,  0.9579, -1.0389],
       [-1.1115,  3.9316,  2.7868]])
```

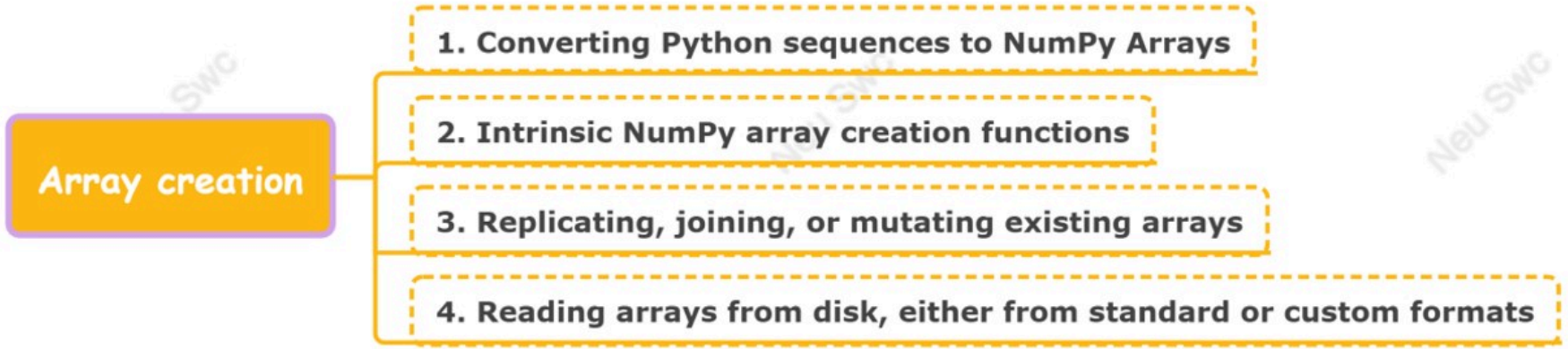

雨课堂 Rain Classroom

The NumPy ndarray

The important attributes of an ndarray object:

attributes	Description
ndim	return int, the number of axes (dimensions维数) of the array.
shape	return tuple(n,m), indicating the size of the array in each dimension.
size	return int(n*m), the total number of elements of the array.
dtype	return data-type, describing the type of the elements in the array.
itemsize	return int, the size in bytes of each element of the array. For example, an array of elements of type float64 has itemsize 8 (=64/8)
flags	Information about the memory layout of the array(有关阵列的内存布局的信息).

Lecture2: NumPy Basics



Array creation

Introduction

- syntax: *numpy.array(object, dtype=None, copy=True, order='K', subok=False, ndmin=0)*

- There are 6 general mechanisms for creating arrays:
  - Conversion from other Python structures (i.e. *lists* and *tuples*)
  - Intrinsic(内部的) NumPy array creation functions (e.g. *arange, ones, zeros*, etc.)
  - Replicating, joining, or mutating existing arrays(复制/连接或改变现有数组)
  - Reading arrays from disk, either from standard or custom formats
  - Creating arrays from raw bytes through the use of strings or buffers
  - Use of special library functions (e.g., random)

Array creation

1. Converting Python sequences to NumPy Arrays

```
>>> import numpy as np                          # 导入NumPy库
>>> arr1D = np.array([1, 2, 3, 4])              # 创建一维数组
>>> print('创建的数组为：', arr1D)
```

```
创建的数组为： [1 2 3 4]
```

```
>>> arr2D = np.array([[1, 2, 3, 4], [4, 5, 6, 7], [7, 8, 9, 10]])   # 创建二维数组
>>> print('创建的数组为：\n', arr2D)
```

```
创建的数组为:
[[ 1  2  3  4]
 [ 4  5  6  7]
 [ 7  8  9 10]]
```

17

雨课堂 Rain Classroom

Array creation

Viewing array attributes

```
>>> print('数组类型为：', arr2D.dtype)
数组类型为： int32
>>> print('数组元素个数为：', arr2D.size)
数组元素个数为： 12
>>> print('数组每个元素大小为：', arr2D.itemsize)
数组每个元素大小为： 4
>>> print('数组维度为：', arr2D.shape)
数组维度为： (3, 4)
```

- 创建的arr2D数组为：
  [[1 2 3 4]
  [4 5 6 7]
  [7 8 9 10]]

Array creation

Viewing array attributes

```
>>> arr2D.shape = 4, 3        # 重新设置 shape
>>> print('重新设置 shape 后的 arr2D 为：\n', arr2D)
```

重新设置shape维度后的arr2为：

```
[[ 1  2  3]
 [ 4  4  5]
 [ 6  7  7]
 [ 8  9 10]]
```

- 原始arr2D数组为：

```
[[ 1  2  3  4]
 [ 4  5  6  7]
 [ 7  8  9 10]]
```


雨课堂 Rain Classroom

Array creation

1. Converting Python sequences to NumPy Arrays

```
>>> import numpy as np                    # 导入NumPy库
>>> arr1D = np.array([1, 2, 3, 4])        # 创建一维数组
>>> print('创建的数组为：', arr1D)
```

创建的数组为： [1 2 3 4]

```
>>> arr2D = np.array([[1, 2, 3, 4],[4, 5, 6, 7], [7, 8, 9, 10]])   # 创建二维数组
>>> print('创建的数组为：\n', arr2D)
```

创建的数组为：
[[1 2 3 4]
[4 5 6 7]
[7 8 9 10]]

Conversion from other Python structures
- low efficiency!
- NumPy provides many Intrinsic functions for creating arrays

Array creation

2. Intrinsic NumPy array creation functions

- NumPy has over 40 built-in functions for creating arrays.
- These functions can be split into roughly three categories, based on the dimension of the array they create:
  - 1D arrays
    - numpy.arange, numpy.linspace, numpy.logspace,
  - 2D arrays
    - numpy.eye, numpy.diag, and numpy.vander
  - ndarrays
    - numpy.ones, numpy.zeros, and random

Array creation

2. Intrinsic NumPy array creation functions

- **1D arrays**

arange([start,] stop[, step,], dtype=None)

```
>>> print('使用 arange 函数创建的等差数列数组为：\n', np.arange(0, 1, 0.1))
```

```
使用 arange 函数创建的等差数列数组为:
[ 0.  0.1  0.2  0.3  0.4  0.5  0.6  0.7  0.8  0.9]
```


雨课堂 Rain Classroom

Array creation

2. Intrinsic NumPy array creation functions

- **1D arrays**

linspace(start, stop, num=50, endpoint=True, retstep=False, dtype=None, axis=0)

```
>>> print('使用 linspace 函数创建的等差数列数组为：\n', np.linspace(1, 4, 6))
```

```
使用 linspace 函数创建的等差数列数组为：
[1. ,  1.6,  2.2,  2.8,  3.4,  4. ]
```


雨课堂 Rain Classroom

Array creation

2. Intrinsic NumPy array creation functions

- **1D arrays**

logspace(start, stop, num=50, endpoint=True, base=10.0, dtype=None, axis=0)

```
>>> print('使用 logspace 函数创建的等比数列数组为：\n', np.logspace(0, 2, 20))
```

```
使用 logspace 函数创建的等比数列数组为：
[   1.          1.27427499    1.62377674 ...,   61.58482111   78.47599704  100.        ]
```

- 在logspace中，起始位和终止位代表的是10的幂（默认基数为10）
  - 0代表10的0次方，2代表10的2次方

雨课堂 Rain Classroom

Array creation

2. Intrinsic NumPy array creation functions

- **2D arrays**

eye(N, M=None, k=0, dtype=<class 'float'>, order='C')

```
>>> print('使用eye函数创建主对角为1的数组：\n', np.eye(3))
```

```
使用eye函数创建主对角为1的数组：
[[ 1.  0.  0.]
 [ 0.  1.  0.]
 [ 0.  0.  1.]]
```

类似单位矩阵。

雨课堂 Rain Classroom

Array creation

2. Intrinsic NumPy array creation functions

- **2D arrays**

diag(v, k=0)

```
>>> print('使用diag函数创建类似对角的数组为：\n', np.diag([1, 2, 3 ,4]))
```

```
使用diag函数创建类似对角的数组为：
[[1 0 0 0]
 [0 2 0 0]
 [0 0 3 0]
 [0 0 0 4]]
```


雨课堂 Rain Classroom

Array creation

2. Intrinsic NumPy array creation functions

- **2D arrays**

vander(x, N=None, increasing=False)

```
>>> x = np.array([1, 2, 3, 5])
>>> N = 3
>>> np.vander(x, N)
```

```
array([[ 1,  1,  1],
       [ 4,  2,  1],
       [ 9,  3,  1],
       [25,  5,  1]])
```

```
>>> x = np.array([1, 2, 3, 5])
>>> np.vander(x)
```

```
array([[  1,   1,   1,   1],
       [  8,   4,   2,   1],
       [ 27,   9,   3,   1],
       [125,  25,   5,   1]])
```

```
>>> x = np.array([1, 2, 3, 5])
>>> np.vander(x, increasing=True)
```

```
array([[  1,   1,   1,   1],
       [  1,   2,   4,   8],
       [  1,   3,   9,  27],
       [  1,   5,  25, 125]])
```

- Vandermonde 矩阵的每一列都是输入一维数组或列表或元组的递减幂， 其中x最高多项式阶数为n-1。
- 此数组创建有助于生成线性最小二乘模型

Array creation

2. Intrinsic NumPy array creation functions

- **ndarrays**

  ones(shape, dtype=None, order='C')

```
>>> print('使用ones函数创建元素皆为1的数组：\n', np.ones((5,3)))
```

```
使用ones函数创建元素皆为1的数组：
[[ 1.  1.  1.]
 [ 1.  1.  1.]
 [ 1.  1.  1.]
 [ 1.  1.  1.]
 [ 1.  1.  1.]]
```

```
>>> np.ones((2,3,2))
```

```
array([[[1., 1.],
        [1., 1.],
        [1., 1.]],

       [[1., 1.],
        [1., 1.],
        [1., 1.]]])
```


雨课堂 Rain Classroom

Array creation

2. Intrinsic NumPy array creation functions

- **ndarrays**

zeros(shape, dtype=float, order='C')

```
>>> print('使用zeros函数创建元素皆为0的数组：\n', np.zeros((2,3)))
使用zeros函数创建元素皆为0的数组：
[[ 0.  0.  0.]
 [ 0.  0.  0.]]
```

```
>>> np.zeros((2,3,2)) )
array([[[0., 0.],
        [0., 0.],
        [0., 0.]],

       [[0., 0.],
        [0., 0.],
        [0., 0.]]])
```


雨课堂 Rain Classroom

Array creation

2. Intrinsic NumPy array creation functions

- Random sampling (**numpy.random**)

```
>>> print('生成无约束条件的0-1之间的随机数组：\n', np.random.random(100))
```

```
生成无约束条件的0-1之间的随机数组：
[0.75946624 0.32403519 0.45156435 0.15301262 0.5466775 0.84605046 0.17080126 0.69076568
0.90944317 0.92458982 0.64758714 0.86632062 0.64711029 0.79839372 0.25877929 0.50130451
0.05525751 0.57771451 0.85348644 0.60981466 0.13559104 0.86325133 0.85926575 0.35266513
0.73434223 0.55986394 0.09632885 0.33584189 0.50353657 0.47070543 0.83600292 0.72432525
0.5941884  0.52633137 0.10449681 0.05879993 0.56039778 0.38883987 0.31812864 0.07824741
0.50415019 0.90873665 0.76296255 0.09991549 0.05827797 0.96862287 0.75540815 0.1081329
0.49204804 0.68595159 0.4948226  0.42642197 0.20368589 0.64483586 0.20556043 0.17892559
0.30848999 0.66396742 0.99889869 0.97470543 0.96726173 0.20558211 0.17261609 0.86407114
0.62561625 0.49406964 0.10720964 0.49829874 0.0886082  0.6681555  0.55979343 0.49613836
0.21340078 0.16162733 0.20244058 0.05463325 0.1685044  0.35679003 0.24781699 0.98330721
0.44642419 0.68006861 0.47913309 0.30670467 0.16266325 0.94817208 0.80733079 0.91267342
0.58278298 0.52347971 0.59467818 0.81292595 0.5379403  0.37008571 0.93050302 0.04367345
0.48252687 0.5820318  0.3328869  0.92678122]
```

Array creation

2. Intrinsic NumPy array creation functions

- Random sampling (**numpy.random**)

```
>>> print('生成服从均匀分布的随机数组：\n', np.random.rand(10, 5))
```

```
生成服从均匀分布的随机数组：
 [[0.49003214 0.95476764 0.80478427 0.02563938 0.73185181]
 [0.83066547 0.86510738 0.33880589 0.81808229 0.23000459]
 [0.28728246 0.12556771 0.9790872  0.79213652 0.94980362]
 [0.83897353 0.18182662 0.90904378 0.00172149 0.54878672]
 [0.50801147 0.67028615 0.92488998 0.71749045 0.72056861]
 [0.61933732 0.53879096 0.56671656 0.4520065  0.69900323]
 [0.90862367 0.13648924 0.55345897 0.77281637 0.26118587]
 [0.51397543 0.48149869 0.5896165  0.84167331 0.57252252]
 [0.02712242 0.53574904 0.02550368 0.33877762 0.19738869]
 [0.10046505 0.60883213 0.71470222 0.37963987 0.53927313]]
```


雨课堂 Rain Classroom

Array creation

2. Intrinsic NumPy array creation functions

- Random sampling (**numpy.random**)

```
>>> print('生成符合正态分布的随机数组：\n', np.random.randn(10, 5))
```

```
生成符合正态分布的随机数组：
 [[ 0.4325805  -0.98168945 -0.5376785   0.53174084 -1.2785329 ]
 [ 1.34082081 -0.25767934  1.722428    0.34583182 -0.45464513]
 [ 0.69941229 -1.28239453 -0.98410001  0.23979938 -0.01257589]
 [ 0.00850335  0.696565    1.5523713   1.54642731 -0.43864744]
 [ 0.00256476 -0.76022471  0.81188766  1.21809275  0.08206837]
 [ 0.22081286 -0.71360927 -0.22065364  0.76769452  0.79808283]
 [ 0.47247358 -1.43741152  0.33592785 -1.56177824 -1.03810669]
 [-0.22089089  1.00881105  0.43125875  0.29081499  1.10204025]
 [-0.91473289  0.82456542 -0.96201081  0.13506441  0.54720484]
 [-0.26405772 -1.36281329  0.90194283 -1.91379344 -1.47086922]]
```


雨课堂 Rain Classroom

Array creation

2. Intrinsic NumPy array creation functions

- Random sampling (**numpy.random**)

```
>>> print('生成限定范围的随机整数数组：\n', np.random.randint(2, 10, size = [2, 5]))
```

```
生成限定范围的随机整数数组：
[[6 6 6 6 8]
[9 6 6 8 4]]
```


雨课堂 Rain Classroom

Array creation

2. Intrinsic NumPy array creation functions

- Random sampling (**numpy.random**)

random.uniform(low=0.0, high=1.0, size=None)

```
>>> np.random.uniform()
0.827455693512018
>>> np.random.uniform(1, 5)
2.93533586182789
>>> np.random.uniform(1, 5, 4)      # 生成一维数组
array([ 3.18487512, 1.40233721, 3.17543152, 4.06933042])
```


雨课堂 Rain Classroom

Array creation

2. Intrinsic NumPy array creation functions

- Random sampling (**numpy.random**)

random.uniform(low=0.0, high=1.0, size=None)

```
>>> np.random.uniform(1, 5, (4,3))          # 生成二维数组

array([[ 2.33083328, 1.592934 , 2.38072 ],
       [ 1.07485686, 4.93224857, 1.42584919],
       [ 3.2667912 , 4.57868281, 1.53218578],
       [ 4.17965117, 3.63912616, 2.83516143]])
```


雨课堂 Rain Classroom

Array creation

3. Replicating, joining, or mutating existing arrays

- Once you have created arrays, you can replicate, join, or mutate those existing arrays to create new arrays.
- When you assign an array or its elements to a new variable, you have to explicitly(明确的) ***numpy.copy*** the array, otherwise the variable is a view into the original array. Consider the following example:

```
a = np.array([1, 2, 3, 4])
b = a[:2]
b += 1
print('a =', a, '\nb =', b)
```

a = [2 3 3 4]

b = [2 3]

```
a = [1, 2, 3, 4]
b = a[:2]
b += 1          # Error
print('a =', a, '\nb =', b)
```

TypeError: 'int' object is not iterable(迭代)

```
a = [1, 2, 3, 4]
b = a[:2]
b[0] += 1
b[1] += 1
print('a =', a, '\nb =', b)
```

a = [1, 2, 3, 4]

b = [2, 3]

```
a = np.array([1, 2, 3, 4])
b = a[:2].copy()
b += 1
print('a =', a, '\nb =', b)
```

a = [1 2 3 4]

b = [2 3]

- 从这个对比看出，numpy和python的区别，~~numpy自动迭代。list这种赋值就算是copy副本了，numpy不算。~~

Array creation

3. Replicating, joining, or mutating existing arrays

- There are a number of routines to join existing arrays e.g. ***numpy.vstack***, ***numpy.hstack***, and ***numpy.block***.
- Here is an example of joining four 2-by-2 arrays into a 4-by-4 array using block:

```
>>> A = np.ones((2, 2))
>>> B = np.eye(2, 2)
>>> C = np.zeros((2, 2))
>>> D = np.diag((-3, -4))
>>> np.block([[A, B], [C, D]])
```

```
array([[ 1.,  1.,  1.,  0.],
       [ 1.,  1.,  0.,  1.],
       [ 0.,  0., -3.,  0.],
       [ 0.,  0.,  0., -4.]])
```


雨课堂 Rain Classroom

Array creation

4. Reading arrays from disk, either from standard or custom formats

- This is the most common case of **large array**(大型数组) **creation**.
- The details depend greatly on the format of data on disk.

(1) Standard Binary Formats

```
np.save("../tmp/save_arr.npy", arr)          # ndarray --> file
np.savez('../tmp/savez_arr.npz', arr1, arr2) # ndarray1, ndarrray2,... --> file
np.load("../tmp/save_arr.npy")               #  file --> ndarray
np.load("../tmp/save_arr.npz")               #  file --> ndarray1, ndarrray2,...
```


雨课堂 Rain Classroom

Array creation

4. Reading arrays from disk, either from standard or custom formats

- This is the most common case of **large array**(大型数组) **creation**.
- The details depend greatly on the format of data on disk.

(2) Common ASCII Formats

```
np.savetxt("../tmp/arr.txt", arr, fmt="%d", delimiter=",")      # ndarray --> ASCII file
np.loadtxt("../tmp/arr.txt", delimiter=",")                     # ASCII file --> ndarray
np.genfromtxt("../tmp/arr.txt", delimiter = ",")                # ASCII file --> ndarray
```


雨课堂 Rain Classroom

4. Reading arrays from disk, either from standard or custom formats

(1) Standard Binary Formats

- *np.save(file, arr, allow_pickle=True, fix_ imports=True)*

```
>>> import numpy as np                          # 导入NumPy库
>>> arr = np.arange(100).reshape(10, 10)        # 创建一个数组
>>> arr
array([[ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14, 15, 16, 17, 18, 19],
       [20, 21, 22, 23, 24, 25, 26, 27, 28, 29],
       [30, 31, 32, 33, 34, 35, 36, 37, 38, 39],
       [40, 41, 42, 43, 44, 45, 46, 47, 48, 49],
       [50, 51, 52, 53, 54, 55, 56, 57, 58, 59],
       [60, 61, 62, 63, 64, 65, 66, 67, 68, 69],
       [70, 71, 72, 73, 74, 75, 76, 77, 78, 79],
       [80, 81, 82, 83, 84, 85, 86, 87, 88, 89],
       [90, 91, 92, 93, 94, 95, 96, 97, 98, 99]])
>>> np.save("../tmp/save_arr", arr)      # 保存数组，文件扩展名.npy自动添加
```


雨课堂 Rain Classroom

4. Reading arrays from disk, either from standard or custom formats

(1) Standard Binary Formats

- *np.savez('../tmp/savez_arr.npz', arr1, arr2)*

```
>>> import numpy as np
>>> arr1 = np.array([[1, 2, 3], [4, 5, 6]])
>>> arr2 = np.arange(0, 1.0, 0.1)
>>> print('保存的数组1为：\n', arr1)
>>> print('保存的数组2为：\n', arr2)
```

```
保存的数组1为:
 [[1 2 3]
 [4 5 6]]
保存的数组2为:
[0.  0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9]
```

```
>>> np.savez('../tmp/savez_arr', arr1, arr2)
```

41

4. Reading arrays from disk, either from standard or custom formats

(1) Standard Binary Formats

- *np.load("../tmp/save_arr.npy")*

```
>>> import numpy as np
>>> loaded_data = np.load("../tmp/save_arr.npy")    # 读取含有单个数组的文件
>>> print('读取的数组为：\n', loaded_data)
```

```
读取的数组为:
[[ 0  1  2  3  4  5  6  7  8  9]
[10 11 12 13 14 15 16 17 18 19]
[20 21 22 23 24 25 26 27 28 29]
[30 31 32 33 34 35 36 37 38 39]
[40 41 42 43 44 45 46 47 48 49]
[50 51 52 53 54 55 56 57 58 59]
[60 61 62 63 64 65 66 67 68 69]
[70 71 72 73 74 75 76 77 78 79]
[80 81 82 83 84 85 86 87 88 89]
[90 91 92 93 94 95 96 97 98 99]]
```


雨课堂 Rain Classroom

4. Reading arrays from disk, either from standard or custom formats

(1) Standard Binary Formats

- *np.load("../tmp/save_arr.npz")*

```
>>> import numpy as np
>>> loaded_data1 = np.load("../tmp/savez_arr.npz")  # 读取含有多个数组的文件
>>> loaded_data1.files     # 查看每个数组的名字
>>> print('读取的数组1为：\n', loaded_data1['arr_0'])
>>> print('读取的数组2为：\n', loaded_data1['arr_1'])
```

```
读取的数组1为:
[[1 2 3]
 [4 5 6]]

读取的数组2为:
[0.  0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9]
```

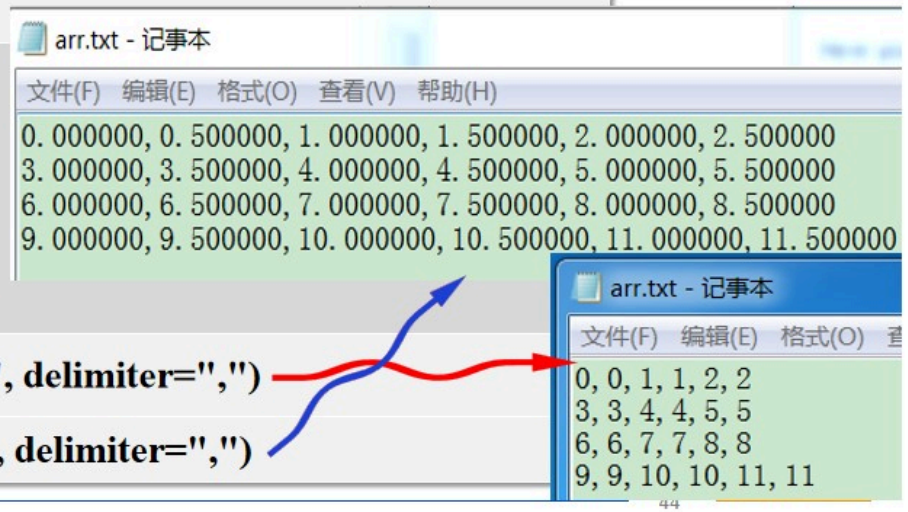
4. Reading arrays from disk, either from standard or custom formats

(2) Common ASCII Formats

- *np.savetxt("../tmp/arr.txt", arr, fmt="%d", delimiter=",")*

```
>>> import numpy as np
>>> arr = np.arange(0, 12, 0.5).reshape(4, -1)
>>> print('创建的数组为：\n', arr)
创建的数组为：
[[ 0.   0.5  1.   1.5  2.   2.5]
 [ 3.   3.5  4.   4.5  5.   5.5]
 [ 6.   6.5  7.   7.5  8.   8.5]
 [ 9.   9.5 10.  10.5 11.  11.5]]
>>> np.savetxt("../tmp/arr.txt", arr, fmt="%d", delimiter=",")
>>> np.savetxt("../tmp/arr.txt", arr, fmt="%f", delimiter=",")
```

Numpy会根据剩下的维度计算出数组的另外一个shape属性值



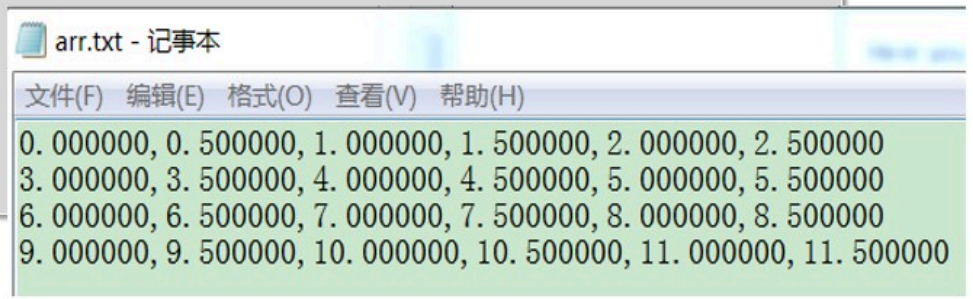
4. Reading arrays from disk, either from standard or custom formats

(2) Common ASCII Formats

- *np.loadtxt("../tmp/arr.txt", delimiter=",")*

```
>>> loaded_data = np.loadtxt("arr.txt", delimiter=",")      # 读入的时候也需要指定逗号分隔
>>> print('读取的数组为：\n', loaded_data)
```

```
读取的数组为:
[[ 0.   0.5  1.   1.5  2.   2.5]
 [ 3.   3.5  4.   4.5  5.   5.5]
 [ 6.   6.5  7.   7.5  8.   8.5]
 [ 9.   9.5 10.  10.5 11.  11.5]]
```

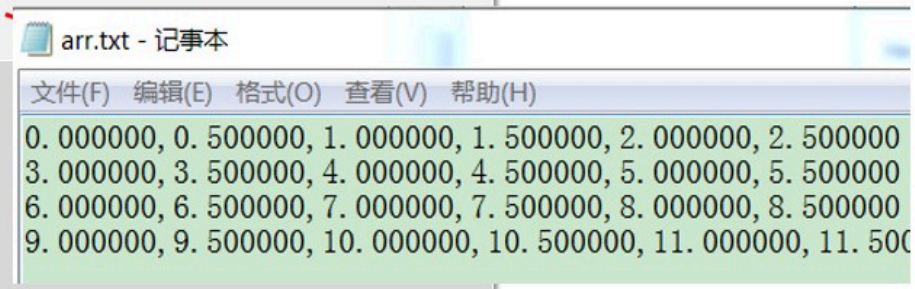


4. Reading arrays from disk, either from standard or custom formats

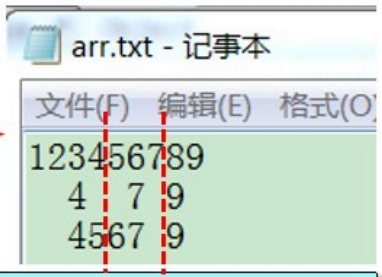
(2) Common ASCII Formats

- *np.genfromtxt("../tmp/arr.txt", delimiter = ",")*

```
>>> loaded_data = np.genfromtxt("arr.txt", delimiter = ",")
>>> print('读取的数组为：\n', loaded_data)
读取的数组为：
[[ 0.   0.5  1.   1.5  2.   2.5]
 [ 3.   3.5  4.   4.5  5.   5.5]
 [ 6.   6.5  7.   7.5  8.   8.5]
 [ 9.   9.5 10.  10.5 11.  11.5]]
```




```
>>> loaded_data2 = np.genfromtxt("arr.txt", delimiter = (4,3,2))
>>> print('读取的数组为：\n', loaded_data2)
读取的数组为：
[[1234.  567.   89.]
 [   4.    7.    9.]
 [  45.   67.    9.]]
```



genfromtxt分割一个固定宽度的文件，列的宽度被定义为一个给定的字符数

NumPy data types

1. What is data type

- The ***data type*** or dtype is a special object containing the information the ndarray needs to interpret a chunk of memory as a particular type of data:

```
>>> arr1 = np.array([1, 2, 3], dtype=np.float64)
>>> arr1.dtype
dtype('float64')
>>> arr2 = np.array([1, 2, 3], dtype=np.int32)
>>> arr2.dtype
dtype('int32')
```

单选题 1分

Python整数结构中包含哪些额外信息?

A 内存地址

B 引用计数

C 类型信息

D 以上皆是

雨课堂 Rain Classroom

单选题 1分

以下哪些是NumPy中用于计算两个向量点积的函数或方法?

A np.multiply()

B np.dot()

C np.sum()

D 以上都可以

雨课堂 Rain Classroom

单选题 1分

以下哪种方式在NumPy中计算数组元素的自然指数更高效?

A 使用for循环逐个计算

B 使用NumPy的exp函数进行向量化计算

单选题 1分

18/3/20

NumPy ndarray 是什么类型的对象?

A 一个用于存储多种数据类型的列表

B 一个用于存储单一数据类型的多维数组

C 一个用于执行数学运算的函数库

D 一个用于数据可视化的工具

多选题 1分

NumPy数组相比Python内置序列有哪些优势?

A 使用更少的内存

B 存储在连续的内存块中

C 有内置的min和max函数

D 算法库用C语言编写，效率高

多选题 1分

以下哪些方式可以用来导入NumPy库?

A import numpy

B import numpy as np

C from numpy import *

D from numpy import array as np_array

多选题 1分

关于Python列表和数组，下列哪些说法是正确的?

A Python列表可以包含不同类型的元素

B Python数组（array模块）只能存储统一类型的数据

C Python列表每个元素都是指向完整Python对象的指针

D array模块支持多维数组和各种运算函数

雨课堂 Rain Classroom

单选题 1分

关于np.linspace函数，下列说法正确的是：

A np.linspace用于创建等差数列数组。

B np.linspace用于创建等比数列数组。

C np.linspace只能创建二维数组。

D np.linspace的默认num参数为100。

雨课堂 Rain Classroom

单选题 1分

以下哪个函数可以生成限定范围的随机整数数组?

A np.random.randint

B np.random.uniform

C np.random.normal

D np.random.choice

单选题 1分

以下哪个函数可以将NumPy数组保存为标准二进制格式?

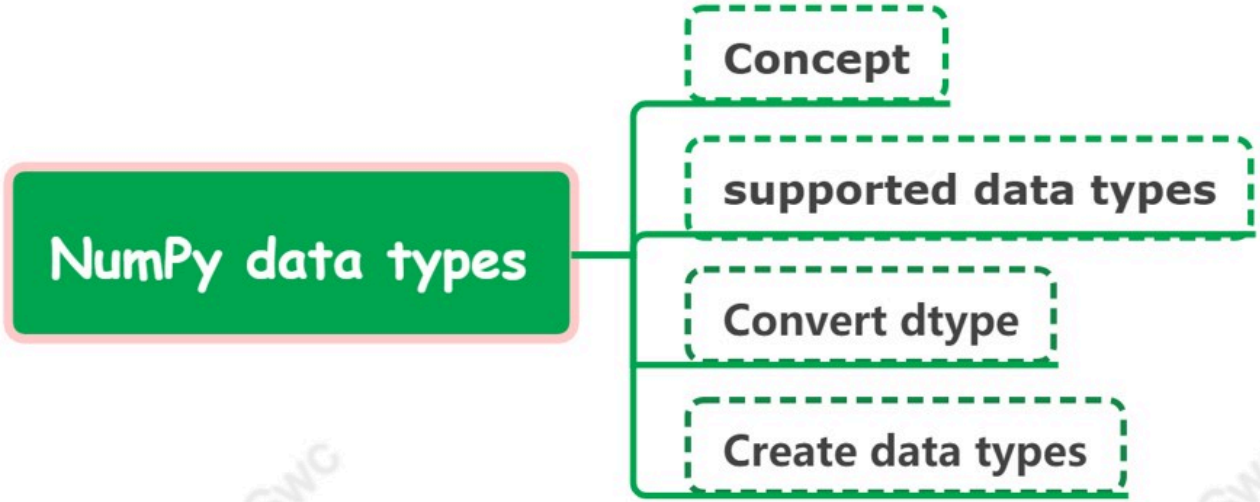
A np.save

B np.savetxt

C np.genfromtxt

D np.load

Lecture2: NumPy Basics



NumPy data types

2. NumPy's supported data types

- The numerical dtypes are named the same way: a type name, like float or int, followed by a number indicating the number of bits per element.

Type	Description
bool	Boolean type storing True and False values
int8, int16, int32, int64	Signed 8-bit/16-bit/32-bit/64-bit integer types
uint8, uint16, uint32, uint64	Unsigned 8-bit/16-bit/32-bit/64-bit integer types
float16, float32, float64, float128	Half-precision/Standard single-precision/double-precision floating point
complex64, complex128, complex256	Complex numbers represented by two 32, 64, or 128 floats, respectively
string_	Fixed-length ASCII string type (1 byte per character); for example, to create a string dtype with length 10, use 'S10'
unicode_	Fixed-length Unicode type (number of bytes platform specific); same specification semantics as string_ (e.g., 'U10')

雨课堂 Rain Classroom

NumPy data types

3. Convert dtype

```
>>> arr = np.array([1, 2, 3, 4, 5])
>>> arr.dtype
dtype('int64')
>>> float_arr = arr.astype(np.float64)     # 整型转换为浮点型
>>> float_arr.dtype
dtype('float64')
>>> arr = np.array([3.7, -1.2, -2.6, 0.5, 12.9, 10.1])
>>> arr.astype(np.int32)                   # 浮点型转换为整型(小数部分被截掉)
array([ 3, -1, -2, 0, 12, 10], dtype=int32)
```

60

雨课堂 Rain Classroom

NumPy data types

3. Convert dtype

```
>>> numeric_strings = np.array(['1.25', '-9.6', '42'], dtype=np.string_)
>>> numeric_strings.dtype
dtype('S4')                                    # 最长字符串长度是4
>>> numeric_strings.astype(float)              # 字符串型转换为浮点型
array([ 1.25, -9.6 , 42. ])
>>> int_array = np.arange(10)                  # 创建整数型数组
>>> calibers = np.array([.22, .270, .357, .380, .44, .50], dtype=np.float64)
>>> int_array.astype(calibers.dtype)          # 使用其他数组的dtype
array([ 0., 1., 2., 3., 4., 5., 6., 7., 8., 9.])
```

61

NumPy data types

4. Create data types

- 创建一个存储餐饮企业库存信息的数据类型。其中
  - 用一个长度为40个字符的字符串来记录商品的名称
  - 用一个64位的整数来记录商品的库存数量
  - 最后用一个64位的双精度浮点数来记录商品的价格，如下：

```
>>> df = np.dtype([("name", np.str_, 40), ("numitems", np.int64), ("price", np.float64)])
>>> print('数据类型为：', df)
```

```
数据类型为：  [('name', '<U40'), ('numitems', '<i8'), ('price', '<f8')]
```

- dtype也可以作为参数，创建特定的类型的数组

NumPy data types

4. Create data types

- 在使用array函数创建数组时，数组的数据类型默认是浮点型。自定义数组数据，则可以预先指定数据类型

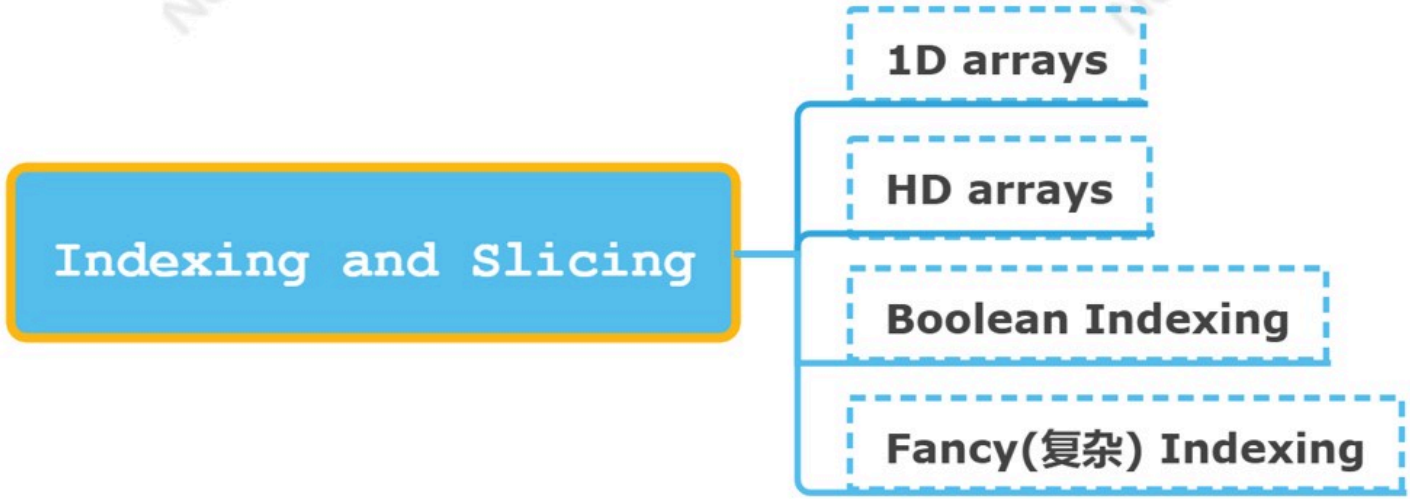
```
>>> itemz = np.array([("tomatoes", 42, 4.14), ("cabbages", 13, 1.72)], dtype=df)
>>> print('自定义数据为：', itemz)
```

```
自定义数据为：  [('tomatoes', 42,  4.14) ('cabbages', 13,  1.72)]
```

```
df = np.dtype([("name", np.str_, 40), ("numitems", np.int64), ("price",np.float64)])
```


雨课堂 Rain Classroom

Lecture2: NumPy Basics



Basic Indexing and Slicing

1. One-dimensional arrays

```
>>> arr = np.arange(10)
>>> arr
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
>>> arr[5]
5
>>> arr[5:8]          #用范围作为索引获取数组的一个切片，包括arr[3]不包括arr[5]
array([5, 6, 7])
>>> arr[:5]           # 省略开始索引，表示从arr[0]开始
array([0 1 2 3 4])
>>> arr[-1]           # 索引可以使用负数，-1表示从数组后往前数的第一个元素
9
```

Basic Indexing and Slicing

1. One-dimensional arrays

```
>>> arr = np.arange(10)
>>> arr
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
>>> arr[2:4] = 100          # 使用索引修改元素的值，且值广播到整个切片
>>> arr
array([  0,   1, 100, 100,   4,   5,   6,   7,   8,   9])
>>> arr[1:-1:2]             # 范围中的第三个参数表示步长，2表示隔一个元素取一个元素
array([  1, 101,   5,   7])
>>> arr[5:1:-2]             # 步长为负数时，开始索引必须大于结束索引
array([  5, 100])
```

66

Basic Indexing and Slicing

1. One-dimensional arrays

```
>>> arr
array([  0,   1, 100, 100,   4,   5,   6,   7,   8,   9])
>>> arr_slice = arr[2:4]          # 赋值arr一个切片到另一个数组arr_slice，它是原始数组的视图
>>> arr_slice                     # 这与Python build-in list是不同的，arr_slice的更改会影响arr
array([ 100, 100])
>>> arr_slice[1] = 200            # 修改切片中的值
>>> arr
array([  0,   1, 100, 200,   4,   5,   6,   7,   8,   9])
>>> arr_slice2 = arr[2:4].copy()  # 获得arr切片的一个副本，它的更改不会改变原始数组的值
>>> arr_slice2 = 300
```

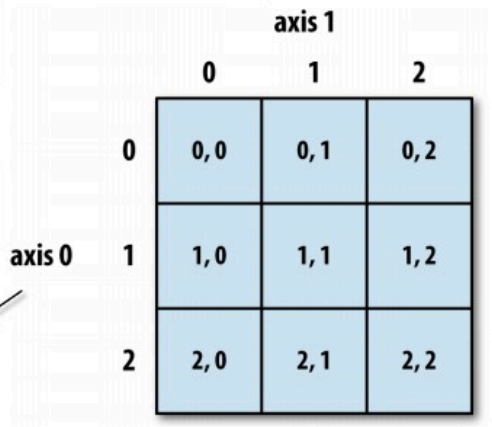

雨课堂 Rain Classroom

Basic Indexing and Slicing

2. higher dimensional arrays

```
>>> arr2D = np.array([[1, 2, 3, 4, 5], [4, 5, 6, 7, 8], [7, 8, 9, 10, 11]])
>>> arr2D
array([[ 1,  2,  3,  4,  5],
       [ 4,  5,  6,  7,  8],
       [ 7,  8,  9, 10, 11]])
arr2D[2]
array([ 7,  8,  9, 10, 11])
arr2D[0, 3:5]                    # 索引第0行中第3和4列的元素
array([4, 5])
```

axis 0 as the "rows" of the array and axis 1 as the "columns."



雨课堂 Rain Classroom

Basic Indexing and Slicing

2. higher dimensional arrays

```
>>> arr2D = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
>>> arr2D
array([[ 1,  2,  3],
       [ 4,  5,  6],
       [ 7,  8,  9]])
arr2D[:2]                    # Indexing with slices
array([[1, 2, 3],
       [4, 5, 6]])
arr2D[:2, 1:]                # Indexing with slices
array([[2, 3],
       [5, 6]])
```

Basic Indexing and Slicing

2. higher dimensional arrays

```
>>> arr3D = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]])      #  2×2×3 array
>>> arr3D
array([[[ 1, 2, 3],
        [ 4, 5, 6]],
       [[ 7, 8, 9],
        [10, 11, 12]]])

>>> arr3D[0]              # arr3d[0] is a 2×3 array
array([[1, 2, 3],
       [4, 5, 6]])

>>> arr3D[1, 0]

array([7, 8, 9])
```


雨课堂 Rain Classroom

Basic Indexing and Slicing

3. Boolean Indexing

```
>>> names = np.array(['Bob', 'Joe', 'Will', 'Bob', 'Will', 'Joe', 'Joe'])
>>> mask = (names == 'Bob')              # mask是一个布尔数组
>>> mask
array([ True, False, False, True, False, False, False])
>>> data = np.random.randn(7, 4)         # 构建测试数据
>>> data
array([[ 0.0929, 0.2817, 0.769 , 1.2464],
       [ 1.0072, -1.2962, 0.275 , 0.2289],
       [ 1.3529, 0.8864, -2.0016, -0.3718],
       [ 1.669 , -0.4386, -0.5397, 0.477 ],
       [ 3.2489, -1.0212, -0.5771, 0.1241],
       [ 0.3026, 0.5238, 0.0009, 1.3438],
       [-0.7135, -0.8312, -2.3702, -1.8608]])
>>> data[mask]
array([[ 0.0929, 0.2817, 0.769 , 1.2464],
       [ 1.669 , -0.4386, -0.5397, 0.477 ]])
>>> data[mask, 2:]
array([[ 0.769 , 1.2464],
       [-0.5397, 0.477 ]])
```


雨课堂 Rain Classroom

Basic Indexing and Slicing

3. Boolean Indexing

```
>>> data                                    # 测试数据
array([[ 0.0929, 0.2817, 0.769 , 1.2464],
       [ 1.0072, -1.2962, 0.275 , 0.2289],
       [ 1.3529, 0.8864, -2.0016, -0.3718],
       [ 1.669 , -0.4386, -0.5397, 0.477 ],
       [ 3.2489, -1.0212, -0.5771, 0.1241],
       [ 0.3026, 0.5238, 0.0009, 1.3438],
       [-0.7135, -0.8312, -2.3702, -1.8608]])
>>> data[data<0]                            # 快速索引到所有小于0的数
array([-1.2962, -2.0016, -0.3718, -0.4386, -0.5397, -1.0212, -0.5771, -0.7135, -0.8312, -2.3702,
-1.8608])
>>> data[data<0] = 0
>>> data
array([[9.2900e-02, 2.8170e-01, 7.6900e-01, 1.2464e+00],
       [1.0072e+00, 0.0000e+00, 2.7500e-01, 2.2890e-01],
       [1.3529e+00, 8.8640e-01, 0.0000e+00, 0.0000e+00],
       [1.6690e+00, 0.0000e+00, 0.0000e+00, 4.7700e-01],
       [3.2489e+00, 0.0000e+00, 0.0000e+00, 1.2410e-01],
       [3.0260e-01, 5.2380e-01, 9.0000e-04, 1.3438e+00],
       [0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00]])
```


雨课堂 Rain Classroom

Basic Indexing and Slicing

4. Fancy(复杂) Indexing

- indexing using integer arrays

```
>>> arr = np.empty((8, 4))        # 创建一个未初始化的二维数组
>>> for i in range(8):
>>>  ..... arr[i] = i              # 初始化数组
>>> arr
array([[ 0., 0., 0., 0.],
       [ 1., 1., 1., 1.],
       [ 2., 2., 2., 2.],
       [ 3., 3., 3., 3.],
       [ 4., 4., 4., 4.],
       [ 5., 5., 5., 5.],
       [ 6., 6., 6., 6.],
       [ 7., 7., 7., 7.]])
```

```
>>> arr[[4, 3, 0, 6]]        # 整型数组索引
array([[ 4., 4., 4., 4.],
       [ 3., 3., 3., 3.],
       [ 0., 0., 0., 0.],
       [ 6., 6., 6., 6.]])

>>> arr[[-3, -5, -7]]
array([[ 5., 5., 5., 5.],
 [ 3., 3., 3., 3.],
 [ 1., 1., 1., 1.]])
```

Basic Indexing and Slicing

4. Fancy(复杂) Indexing

- indexing using integer arrays

```
>>> arr = np.arange(32).reshape((8, 4))      # 创建二维数组
>>> arr
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11],
       [12, 13, 14, 15],
       [16, 17, 18, 19],
       [20, 21, 22, 23],
       [24, 25, 26, 27],
       [28, 29, 30, 31]])
```

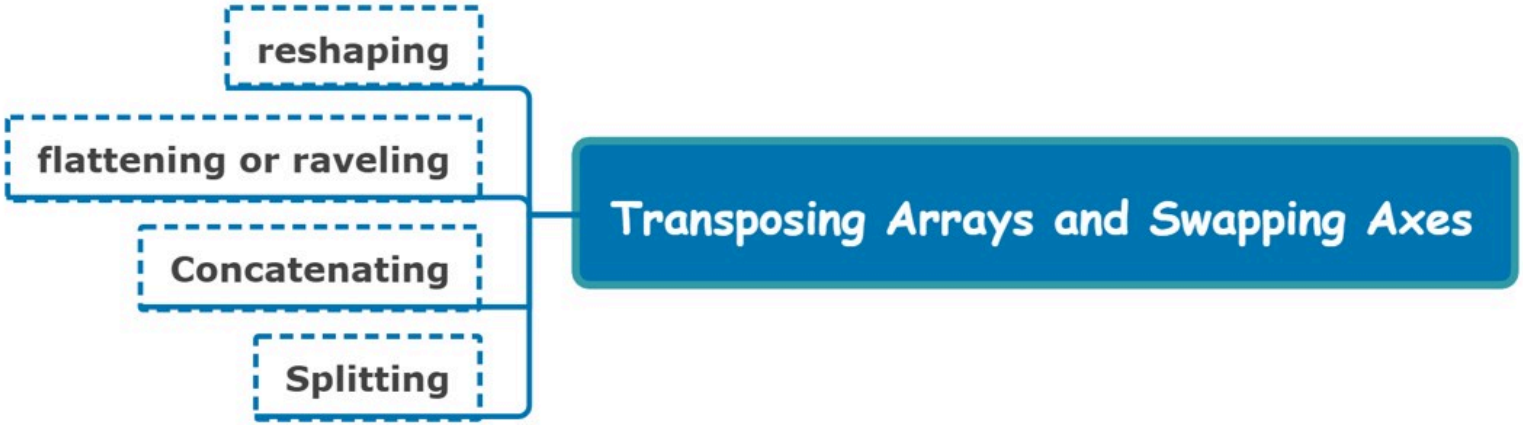
```
>>> arr[[1, 5, 7, 2], [0, 3, 1, 2]]
array([ 4, 23, 29, 10])
>>> arr[[1, 5, 7, 2]][:, [0, 3, 1, 2]]
array([[ 4,  7,  5,  6],
       [20, 23, 21, 22],
       [28, 31, 29, 30],
       [ 8, 11,  9, 10]])
```

What is the value of *arr* after the following operations:

- **arr[[1, 5, 7, 2], [0, 3, 1, 2]] = 100**
- **arr[[1, 5, 7, 2]][:, [0, 3, 1, 2]] = 200**

74

Lecture2: NumPy Basics



Transposing Arrays and Swapping Axes

1. reshaping arrays

```
>>> arr = np.arange(12)              # 创建一维数组
>>> arr
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
>>> arr2D = arr.reshape(3,4)         # 设置数组的形状，但arr本身没变
>>> arr2D
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
>>> arr2D.T                          # 转置
array([[ 0,  4,  8],
       [ 1,  5,  9],
       [ 2,  6, 10],
       [ 3,  7, 11]])
```


雨课堂 Rain Classroom

Transposing Arrays and Swapping Axes

2. flattening or raveling

ndarray.ravel([order]) *ndarray.flatten(order='C')*

Parameters: order{'C', 'F', 'A', 'K'}, optional

```
>>> arr = np.arange(12).reshape(3,4)
>>> arr
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])

>>> arr.ravel()        # Return a contiguous flattened array
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])

>>> arr.flatten()      # Return a copy of the array collapsed into one dimension.
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
```

What is the value of *arr* after the following operations:

- arr.ravel()[3] = 100
- arr.flatten()[4] = 200

77

Transposing Arrays and Swapping Axes

3. Concatenating Arrays

- ***numpy.concatenate*** takes a sequence (tuple, list, etc.) of arrays and joins them together in order along the input axis:
  - *np.concatenate((arr1,arr2), axis = 0))* # 纵向连接
  - *np.concatenate((arr1,arr2), axis = 1))* # 横向连接

- There are some convenience functions, like ***vstack*** and ***hstack***, for common kinds of concatenation.
  - *np.vstack((arr1,arr2))* # 纵向连接
  - *np.hstack((arr1,arr2))* # 横向连接

雨课堂 Rain Classroom

Transposing Arrays and Swapping Axes

3. Concatenating Arrays

- ***numpy.concatenate*** takes a sequence (tuple, list, etc.) of arrays and joins them together in order along the input axis:

```
>>> arr1 = np.arange(6).reshape(2, 3)
>>> arr1
array([[0, 1, 2],
       [3, 4, 5]])
>>> arr2 = np.arange(6,12).reshape(2, 3)
>>> arr2
array([[ 6,  7,  8],
       [ 9, 10, 11]])
```

```
>>> np.concatenate([arr1, arr2], axis=0)
array([[ 1, 2, 3],
       [ 4, 5, 6],
       [ 7, 8, 9],
       [10, 11, 12]])
>>> np.concatenate([arr1, arr2], axis=1)
array([[ 1, 2, 3, 7, 8, 9],
       [ 4, 5, 6, 10, 11, 12]])
```

Transposing Arrays and Swapping Axes

3. Concatenating Arrays

- There are some convenience functions, like ***vstack*** and ***hstack***, for common kinds of concatenation.

```
>>> arr1 = np.arange(6).reshape(2, 3)
>>> arr1
array([[0, 1, 2],
       [3, 4, 5]])
>>> arr2 = np.arange(6,12).reshape(2, 3)
>>> arr2
array([[ 6,  7,  8],
       [ 9, 10, 11]])
```

```
>>> np.vstack((arr1, arr2))
array([[ 1, 2, 3],
       [ 4, 5, 6],
       [ 7, 8, 9],
       [10, 11, 12]])
>>> np.hstack((arr1, arr2))
array([[ 1, 2, 3, 7, 8, 9],
       [ 4, 5, 6, 10, 11, 12]])
```


雨课堂 Rain Classroom

Transposing Arrays and Swapping Axes

4. Splitting Arrays

- split, on the other hand, slices apart an array into multiple arrays along an axis:

```
>>> arr = np.random.randn(4, 2)
>>> arr
array([[-0.99913375,  1.53092114],
       [-1.60934846, -1.02730753],
       [ 0.54534647,  1.29051413],
       [-0.42844545,  0.3828628 ]])
>>> arr1, arr2 = np.split(arr, 2)
>>> arr1
array([[-0.99913375,  1.53092114],
       [-1.60934846, -1.02730753]])
>>> arr2
array([[ 0.54534647,  1.29051413],
       [-0.42844545,  0.3828628 ]])
```

```
>>> arr = np.random.randn(5, 2)
>>> arr
array([[-0.2047, 0.4789],
       [-0.5194, -0.5557],
       [ 1.9658, 1.3934],
       [ 0.0929, 0.2817],
       [ 0.769 , 1.2464]])
>>> first, second, third = np.split(arr, [1, 3])
>>> print(first, '\n', second, '\n', third)
[[-0.2047 0.4789]]
[[-0.5194 -0.5557]
 [ 1.9658 1.3934]]
[[ 0.0929 0.2817]
 [ 0.769 1.2464]]
```

Transposing Arrays and Swapping Axes

4. Splitting Arrays

- split, on the other hand, slices apart an array into multiple arrays along an axis:

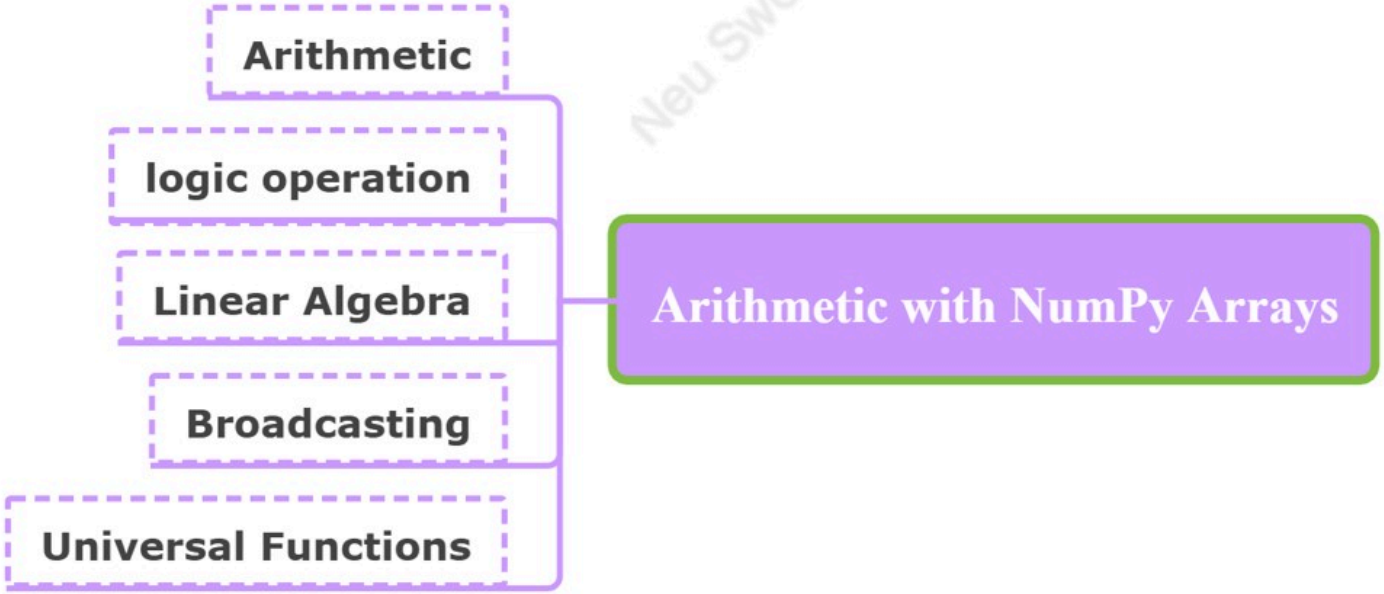
```
>>> arr = np.random.randn(4, 2)
>>> arr
array([[-0.99913375,  1.53092114],
       [-1.60934846, -1.02730753],
       [ 0.54534647,  1.29051413],
       [-0.42844545,  0.3828628 ]])
```

```
>>> np.split(arr, 2, axis=1)
[array([[-0.99913375],
        [-1.60934846],
        [ 0.54534647],
        [-0.42844545]]),
array([[1.53092114],
       [-1.02730753],
       [1.29051413],
       [0.3828628 ]])]
```

```
>>> np.vsplit(arr, 2)
[array([[-0.99913375,  1.53092114],
        [-1.60934846, -1.02730753]]),
 array([[ 0.54534647,  1.29051413],
        [-0.42844545,  0.3828628 ]])]
```

```
>>> np.hsplit(arr, 2)
[array([[-0.99913375],
        [-1.60934846],
        [ 0.54534647],
        [-0.42844545]]),
array([[1.53092114],
       [-1.02730753],
       [1.29051413],
       [0.3828628 ]])]
```

Lecture2: NumPy Basics



Arithmetic with NumPy Arrays

1. Arithmetic

- Arrays are important because they enable you to express batch operations(批量操作) on data without writing any for loops. NumPy users call this *vectorization*(向量化).
- Any arithmetic operations between equal-size arrays applies the operation element-wise(元素级):

```
>>> arr = np.array([[1., 2., 3.], [4., 5., 6.]])
>>> arr
array([[ 1., 2., 3.],
       [ 4., 5., 6.]])
>>> arr + arr
array([[ 2.,  4.,  6.],
       [ 8., 10., 12.]])
>>> arr - arr
array([[ 0. , 0. , 0.],
       [ 0. , 0. , 0.]])
```

```
>>> arr * arr
array([[  1.,  4.,  9.],
       [ 16., 25., 36.]])
>>> 1 / arr
array([[ 1. ,    0.5 ,  0.3333],
       [ 0.25 , 0.2 ,  0.1667]])
>>> arr ** 0.5
array([[ 1. , 1.4142, 1.7321],
       [ 2. , 2.2361, 2.4495]])
```

Arithmetic with NumPy Arrays

2. logic operation

- Comparisons between arrays of the same size yield boolean arrays: >, <, ==, >=, <=, !=

```
>>> arr = np.array([[1., 2., 3.], [4., 5., 6.]])
>>> arr
array([[ 1., 2., 3.],
       [ 4., 5., 6.]])
>>> arr2 = np.array([[0., 4., 1.], [7., 2., 12.]])
>>> arr2
array([[ 0.,  4.,  1.],
       [ 7.,  2., 12.]])
>>> arr2 > arr
array([[False, True, False],
       [True, False, True]], dtype=bool)
```

```
>>> x = np.array([1,3,5])
>>> y = np.array([2,3,4])
>>> x < y
array([ True, False, False])
>>> x > y
array([False, False,  True])
>>> x == y
array([False,  True, False])
>>> x >= y
array([False,  True,  True])
```

Arithmetic with NumPy Arrays

3. Linear Algebra

- Linear algebra, like matrix multiplication(矩阵乘法), decompositions(矩阵分解), determinants(行列式), and other square matrix math(方阵算术), is an important part of any array library.

```
>>> x = np.array([[1., 2., 3.], [4., 5., 6.]])
>>> x
array([[ 1., 2., 3.],
       [ 4., 5., 6.]])
>>> y = np.array([[6., 23.], [-1, 7], [8, 9]])
>>> y
array([[ 6., 23.],
       [ -1., 7.],
       [ 8., 9.]])
```

```
>>> x.dot(y)        # 矩阵乘法
array([[ 28., 64.],
       [ 67., 181.]])
>>> np.dot(x, y)
array([[ 28., 64.],
       [ 67., 181.]])
>>> np.dot(x, np.ones(3))
array([ 6., 15.])
```

Arithmetic with NumPy Arrays

3. Linear Algebra

- *numpy.linalg* has a standard set of matrix decompositions(矩阵分解) and things like inverse(求逆) and determinant(行列式).

```
>>> from numpy.linalg import inv, qr
>>> X = np.random.randn(5, 5)
>>> mat = X.T.dot(X)
>>> inv(mat)        # 求逆矩阵
array([[ 933.1189, 871.8258, -1417.6902, -1460.4005, 1782.1391],
       [ 871.8258, 815.3929, -1325.9965, -1365.9242, 1666.9347],
       [-1417.6902, -1325.9965, 2158.4424, 2222.0191, -2711.6822],
       [-1460.4005, -1365.9242, 2222.0191, 2289.0575, -2793.422 ],
       [ 1782.1391, 1666.9347, -2711.6822, -2793.422 , 3409.5128]])
mat.dot(inv(mat))
array([[ 1., 0., -0., -0., -0.],
 [-0., 1., 0., 0., 0.],
 [ 0., 0., 1., 0., 0.],
 [-0., 0., 0., 1., -0.],
 [-0., 0., 0., 0., 1.]])
>>> q, r = qr(mat)
>>> r
array([[-1.6914, 4.38 , 0.1757, 0.4075, -0.7838],
       [ 0. , -2.6436, 0.1939, -3.072 , -1.0702],
       [ 0. , 0. , -0.8138, 1.5414, 0.6155],
       [ 0. , 0. , 0. , -2.6445, -2.1669],
       [ 0. , 0. , 0. , 0. , 0.0002]])
```


雨课堂 Rain Classroom

Arithmetic with NumPy Arrays

3. Linear Algebra

- Commonly used numpy.linalg functions

Function	Description
`diag`	Return the diagonal (or off-diagonal) elements of a square matrix as a 1D array, or convert a 1D array into a square matrix with zeros on the off-diagonal
`dot`	Matrix multiplication
`trace`	Compute the sum of the diagonal elements
`det`	Compute the matrix determinant

Arithmetic with NumPy Arrays

3. Linear Algebra

- Commonly used numpy.linalg functions

Function	Description
`eig`	Compute the eigenvalues and eigenvectors of a square matrix
`inv`	Compute the inverse of a square matrix
`pinv`	Compute the Moore-Penrose pseudo-inverse of a matrix
`qr`	Compute the QR decomposition
`svd`	Compute the singular value decomposition (SVD)
`solve`	Solve the linear system Ax = b for x, where A is a square matrix
`lstsq`	Compute the least-squares solution to Ax = b

Arithmetic with NumPy Arrays

4. Broadcasting

- Broadcasting describes how arithmetic works between arrays of different shapes.
- It can be a powerful feature, but one that can cause confusion, even for experienced users.
- The simplest example of broadcasting occurs when combining a scalar(标量) value with an array:

```
>>> arr = np.arange(5)
>>> arr
array([0, 1, 2, 3, 4])
>>> arr * 4
array([ 0, 4, 8, 12, 16])
```

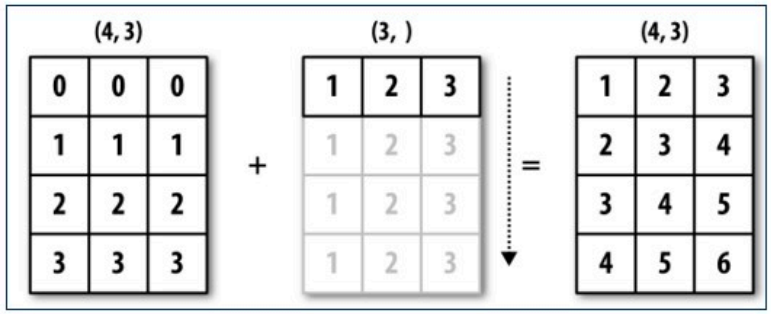
- Here we say that the scalar value 4 has been **broadcast** to all of the other elements in the multiplication operation.

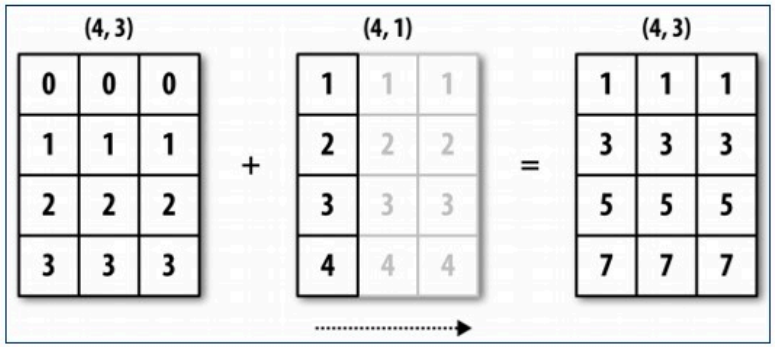
雨课堂 Rain Classroom

Arithmetic with NumPy Arrays

4. Broadcasting

- The Broadcasting Rule
  - Two arrays are compatible for broadcasting if for each *trailing dimension* (i.e., starting from the end) the axis lengths match or if either of the lengths is 1.
  - Broadcasting is then performed over the missing or length 1 dimensions.





Arithmetic with NumPy Arrays

4. Broadcasting

- For example, we can *demean*(去均值) each column of an array by subtracting the column means.
- In this case, it is very simple:

```
>>> arr = np.random.randn(4, 3)
>>> arr
array([[-0.44349445,  0.67889817,  0.37153362],
       [ 0.73519857,  1.80496825,  0.01079588],
       [ 0.0401694 , -1.23455595, -0.16782545],
       [ 0.20064658,  0.31700855, -0.42176051]])
>>> arr.mean(0)        # 每一列求均值
array([ 0.13313003,  0.39157975, -0.05181412])
```

```
>>> demeaned = arr - arr.mean(0)        # 纵向广播
>>> demeaned
array([[-0.57662447,  0.28731842,  0.42334773],
       [ 0.60206854,  1.41338849,  0.06260999],
       [-0.09296062, -1.6261357 , -0.11601134],
       [ 0.06751656, -0.07457121, -0.36994639]])
>>> demeaned.mean(0)        # 每一列求均值
array([-0.,  0.,  0.])
```

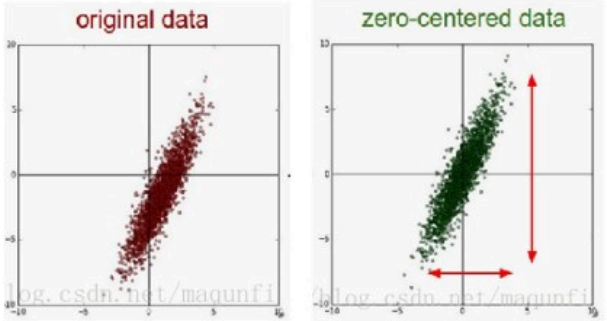

雨课堂 Rain Classroom

补充：去均值化

- **去均值化定义**：各维度都减对应维度的均值，使得输入数据各个维度都中心化为0
- **去均值化目的**：进行去均值的原因是方便快速拟合目标
- 以图像数据为例：
  - 去均值是为了对图像进行标准化，可以移除图像的平均亮度值 (intensity)。很多情况下我们对图像的照度并不感兴趣，而更多地关注其内容。
  - 比如在对象识别任务中，图像的整体明亮程度并不会影响图像中存在的是什么物体。在每个样本上减去数据的统计平均值可以移除共同的部分，凸显个体差异。

(a) 原图

(b) 减均值后



Arithmetic with NumPy Arrays

4. Broadcasting

- For example, we can demean each column of an array by subtracting the column means.
- In this case, it is very simple:

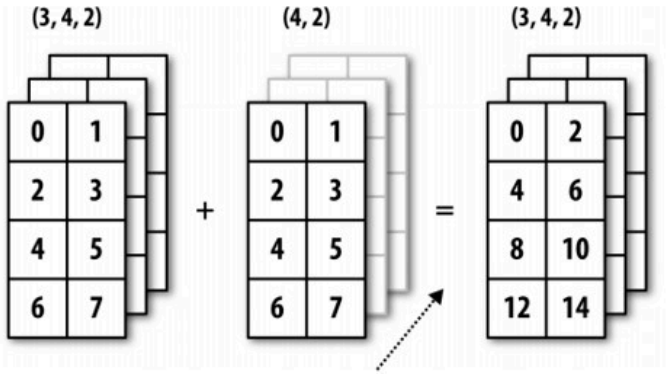
```
>>> arr = np.random.randn(4, 3)
>>> arr
array([[-0.44349445,  0.67889817,  0.37153362],
       [ 0.73519857,  1.80496825,  0.01079588],
       [ 0.0401694 , -1.23455595, -0.16782545],
       [ 0.20064658,  0.31700855, -0.42176051]])
>>> row_means = arr.mean(1)        # 每一行求均值
array([ 0.20231245,  0.8503209 , -0.45407067,  0.03196487])
>>> row_means.shape
(4,)
```

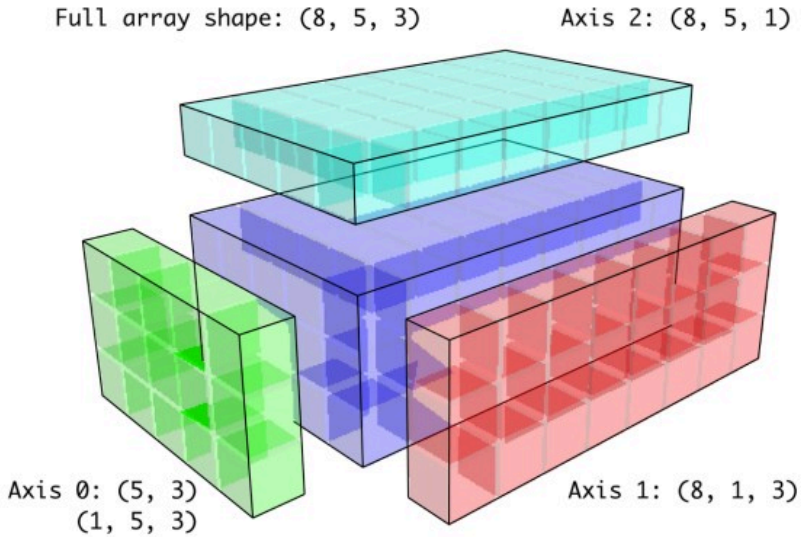
```
>>> row_means.reshape((4, 1))
array([[ 0.20231245],
       [ 0.8503209 ],
       [-0.45407067],
       [ 0.03196487]])
>>> demeaned = arr - row_means.reshape((4, 1))
>>> demeaned
array([[-0.64580689,  0.47658572,  0.16922117],
       [-0.11512233,  0.95464735, -0.83952502],
       [ 0.49424007, -0.78048529,  0.28624521],
       [ 0.16868171,  0.28504367, -0.45372538]])
>>> demeaned.mean(1)        # 每一行求均值
array([0.,  -0.,  0.,  0.])
```

Arithmetic with NumPy Arrays

5. Broadcasting Over Other Axes

- Broadcasting with higher dimensional arrays can seem even more mind-bending, but it is really a matter of following the rules.





Arithmetic with NumPy Arrays

5. Broadcasting Over Other Axes

```
>>> arr_1d = np.random.normal(size=3)
>>> arr_1d
array([ 0.13823199, -2.29133504, -1.17021069])
>>> arr_1d[:, np.newaxis]    # 增加一轴/None别称
array([[ 0.13823199],
       [-2.29133504],
       [-1.17021069]])
>>> arr_1d[np.newaxis, :]    # 增加一轴
array([[ 0.13823199, -2.29133504, -1.17021069]])
```

思考题：如何构建任一轴向为一列的三维数组?

```
>>> arr_2d = np.zeros((4, 4))
>>> arr_2d
array([[[0., 0., 0., 0.]],
       [[0., 0., 0., 0.]],
       [[0., 0., 0., 0.]],
       [[0., 0., 0., 0.]]])
>>> arr_3d = arr_2d[:, np.newaxis, :]
>>> arr_3d
array([[[0., 0., 0., 0.]],

       [[0., 0., 0., 0.]],

       [[0., 0., 0., 0.]],

       [[0., 0., 0., 0.]]])
>>> arr_3d.shape
(4, 1, 4)
```

Arithmetic with NumPy Arrays

5. Broadcasting Over Other Axes

- if we had a three-dimensional array and wanted to demean axis 2:

```
>>> arr_3d = np.random.randn(3, 4, 5)
>>> arr_3d

array([[[ 0.75177074,  0.10149589,  2.33910744,  1.27041306, -1.02902057],
        [-0.73682703, -0.26506733, -0.94742391,  1.33039276,  0.81209546],
        [ 0.73716165,  0.32485195,  0.38810285, -0.91463865,  0.49199111],
        [-0.05662837,  0.39861359, -0.79354059, -0.07678008,  0.63946106]],

       [[-1.18886019,  1.39059244, -0.25370042, -0.02663279, -0.28247252],
        [-1.11832461, -0.55016646,  0.07083715, -0.00604801, -1.55288619],
        [ 0.20305928,  0.47171643, -2.00647536,  0.11585776,  1.19049551],
        [ 1.95470592,  0.76400956, -0.21928781, -2.24490545, -0.36076451]],

       [[ 0.62650848, -1.1647009 , -0.28965058,  1.78430006, -0.17606433],
        [ 0.71968742, -0.09386181,  0.3155264 ,  0.10475855,  1.59412851],
        [-1.45709537, -1.50957047, -1.5255692 ,  0.36678568,  0.33864655],
        [-0.36884651, -0.28689448, -1.08318929, -0.03100872, -0.71414963]]])
```

```
>>> depth_means = arr_3d.mean(2)
>>> depth_means

array([[ 0.68675331,  0.03863399,  0.20549378,  0.02222512],
       [-0.0722147 , -0.63131762, -0.00506928, -0.02124846],
       [ 0.15607855,  0.52804781, -0.75736056, -0.49681773]])
```

```
>>> depth_means.shape

(3, 4)
```

```
>>> demeaned = arr_3d - depth_means

ValueError: operands could not be broadcast together with shapes (3,4,5) (3,4)
```

```
>>> demeaned = arr_3d - depth_means[:, :, np.newaxis]
>>> demeaned.shape

(3, 4, 5)
```

思考题：数组的shape?

雨课堂 Rain Classroom

Arithmetic with NumPy Arrays

6. Setting Array Values by Broadcasting

- The same broadcasting rule governing arithmetic operations also applies to setting values via array indexing. In a simple case, we can do things like:

```
>>> arr = np.zeros((4, 3))
>>> arr
array([[0., 0., 0.],
       [0., 0., 0.],
       [0., 0., 0.],
       [0., 0., 0.]])
>>> arr[:] = 5          # 广播方式赋值
>>> arr
array([[ 5., 5., 5.],
 [ 5., 5., 5.],
 [ 5., 5., 5.],
 [ 5., 5., 5.]])
```

```
>>> col = np.array([1.28, -0.42, 0.44, 1.6])
>>> arr[:] = col[:, np.newaxis]      # 右侧shape=(4, 1)
>>> arr                              # 左侧shape=(4, 3)
array([[ 1.28,  1.28,  1.28],
       [-0.42, -0.42, -0.42],
       [ 0.44,  0.44,  0.44],
       [ 1.6 ,  1.6 ,  1.6 ]])
>>> arr[:2] = [[-1.37], [0.509]]     # 左侧shape=(2, 3)
>>> arr                              # 右侧shape=(2, 1)
array([[-1.37 , -1.37 , -1.37 ],
       [ 0.509,  0.509,  0.509],
       [ 0.44 ,  0.44 ,  0.44 ],
       [ 1.6  ,  1.6  ,  1.6  ]])
```

如果我们有一个一维数组，想要设置这个一维数组中的值到一个数组的每个列中，可以这样做。

Universal Functions: Fast Element-Wise Array Functions

- A universal function, or ***ufunc***, is a function that performs element-wise operations on data in ndarrays.
- You can think of them as fast vectorized(向量化) wrappers for simple functions that take one or more scalar(标量) values and produce one or more scalar results.
- **Example:** Many ufuncs are simple element-wise transformations, like sqrt or exp:

```
>>> arr = np.arange(10)
>>> arr
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
>>> np.sqrt(arr)
array([ 0. , 1. , 1.4142, 1.7321, 2. , 2.2361,
        2.4495, 2.6458, 2.8284, 3. ])
>>> np.exp(arr)
array([ 1. , 2.7183, 7.3891, 20.0855, 54.5982, 148.4132,
403.4288, 1096.6332, 2980.958 , 8103.0839])
```

■ These are referred to as unary ufuncs(一元函数)

雨课堂 Rain Classroom

Universal Functions: Fast Element-Wise Array Functions

- Such as add or maximum, take two arrays (thus, binary ufuncs 二元函数) and return a single array as the result:

```
>>> x = np.random.randn(8)
>>> x
array([-0.0119, 1.0048, 1.3272, -0.9193, -1.5491, 0.0222,  0.7584,  -0.6605])
>>> y = np.random.randn(8)
>>> y
array([ 0.8626, -0.01 ,  0.05 ,  0.6702,  0.853 , -0.9559, -0.0235, -2.3042])
>>> np.maximum(x, y)
array([ 0.8626, 1.0048, 1.3272, 0.6702,  0.853 , 0.0222,  0.7584,  -0.6605])
```

■ These are referred to as binary ufuncs(二元函数)

Universal Functions: Fast Element-Wise Array Functions

- While not common, a ufunc can return multiple arrays. modf is one example, a vectorized version of the built-in Python divmod; it returns the fractional and integral parts of a floating-point array:

```
>>> arr = np.random.randn(7) * 5        # 生成带有整数部分和小数部分的数组
>>> arr
array([-3.2623, -6.0915, -6.663 , 5.3731, 3.6182, 3.45 , 5.0077])
>>> remainder, whole_part = np.modf(arr)
>>> remainder        # 小数部分
array([-0.2623, -0.0915, -0.663 , 0.3731, 0.6182, 0.45 , 0.0077])
>>> whole_part        # 整数部分
array([-3., -6., -6., 5., 3., 3., 5.])
```


雨课堂 Rain Classroom

Universal Functions: Fast Element-Wise Array Functions

- Ufuncs accept an optional *out* argument that allows them to operate **in-place** on arrays:

```
>>> arr
array([-3.2623, -6.0915, -6.663 , 5.3731, 3.6182, 3.45 , 5.0077])
>>> np.sqrt(arr)
array([ nan, nan, nan, 2.318 , 1.9022, 1.8574, 2.2378])
>>> arr
array([-3.2623, -6.0915, -6.663 , 5.3731, 3.6182, 3.45 , 5.0077])      # arr没变
>>> np.sqrt(arr, arr)      # 第二个arr是可选的输出参数
array([ nan, nan, nan, 2.318 , 1.9022, 1.8574, 2.2378])
>>> arr
array([ nan, nan, nan, 2.318 , 1.9022, 1.8574, 2.2378])      # arr变了
```

102

Universal Functions: Fast Element-Wise Array Functions

- Unary ufuncs(一元函数):

Function	Description	
abs, fabs	Compute the absolute value element-wise for integer, floating-point, or complex values	整数浮点数绝对值
sqrt	Compute the square root of each element (equivalent to arr ** 0.5)	平方根
square	Compute the square of each element (equivalent to arr ** 2)	平方
exp	Compute the exponent e^x of each element	e^x
sign	Compute the sign of each element: 1 (positive), 0 (zero), or –1 (negative)	返回每个元素的符号
ceil	Compute the ceiling of each element (i.e., the smallest integer greater than or equal to that number)	每个元素的上限
floor	Compute the floor of each element (i.e., the largest integer less than or equal to each element)	每个元素的下限
rint	Round elements to the nearest integer, preserving the dtype	舍入到最接近的整数
modf	Return fractional and integral parts of array as a separate array	返回整数和小数部分
isnan	Return boolean array indicating whether each value is NaN (Not a Number)	返回布尔数组
isfinite, isinf	Return boolean array indicating whether each element is finite (non-inf, non-NaN) or infinite	返回布尔数组
log, log10, log2, log1p	Natural logarithm (base e), log base 10, 2, and log(1 + x), respectively	对数
cos, cosh, sin, sinh, tan, tanh	Regular and hyperbolic trigonometric functions	三角函数
arccos, arccosh, arcsin, arcsinh, arctan, arctanh	Inverse trigonometric functions	反三角函数
logical_not	Compute truth value of not x element-wise (equivalent to ~arr)	求逻辑反

雨课堂 Rain Classroom

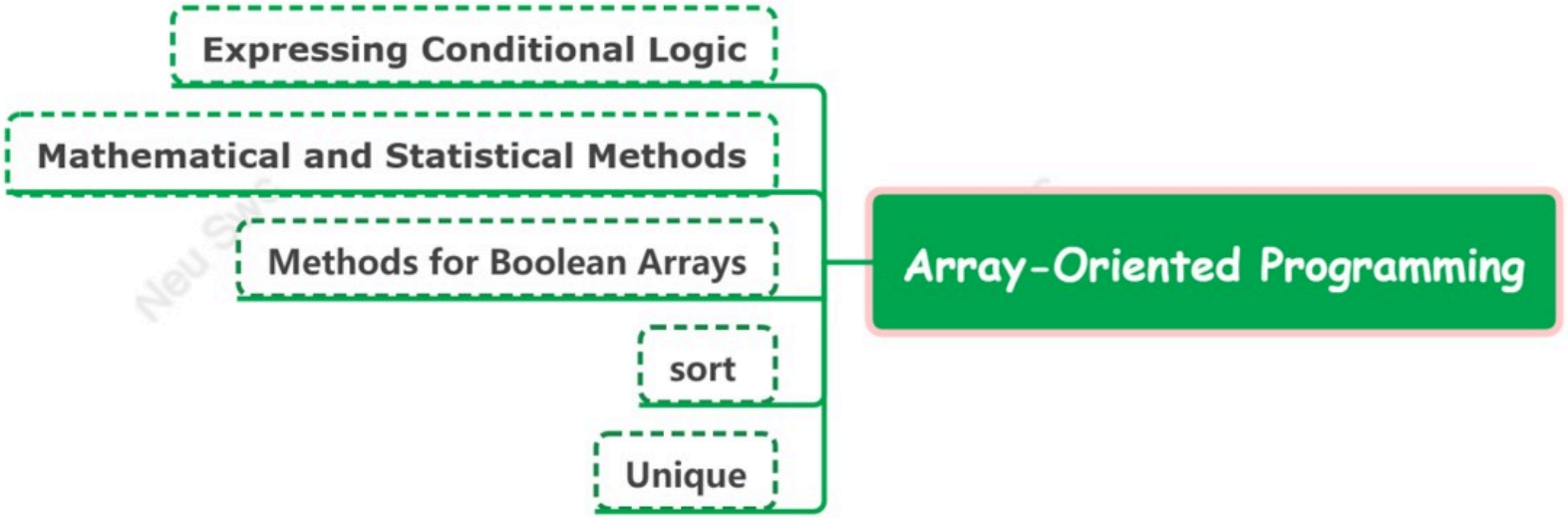
Universal Functions: Fast Element-Wise Array Functions

- Binary universal functions(二元函数):

Function	Description	
add	Add corresponding elements in arrays	加
subtract	Subtract elements in second array from first array	减
multiply	Multiply array elements	乘
divide, floor_divide	Divide or floor divide (truncating the remainder)	除或截断余数除
power	Raise elements in first array to powers indicated in second array	幂运算
maximum, fmax	Element-wise maximum; fmax ignores NaN	求最大值，忽略NaN
minimum, fmin	Element-wise minimum; fmin ignores NaN	求最小值，忽略NaN
mod	Element-wise modulus (remainder of division)	元素级求模(余数)
copysign	Copy sign of values in second argument to values in first argument	第二个参数符号给第一个参数
greater, greater_equal, less, less_equal, equal, not_equal	Perform element-wise comparison, yielding boolean array (equivalent to infix operators >, >=, <, <=, ==, !=)	元素级比较
logical_and, logical_or, logical_xor	Compute element-wise truth value of logical operation (equivalent to infix operators & |, ^)	元素级逻辑运算

雨课堂 Rain Classroom

Lecture2: NumPy Basics



Array-Oriented Programming with Arrays

- Using NumPy arrays enables you to express many kinds of data processing tasks as concise(简洁的) array expressions that might otherwise require writing loops.
- This practice of replacing explicit(显式) loops with array expressions is commonly referred to as vectorization(向量化).
- In general, vectorized array operations will often be one or two (or more) orders of magnitude(数量级) faster than their pure Python equivalents, with the biggest impact(影响) in any kind of numerical computations(数值计算).

雨课堂 Rain Classroom

Array-Oriented Programming with Arrays

- Example:
  - suppose we wished to evaluate the function ***sqrt(x^2 + y^2)*** across a regular grid of values.
  - The ***np.meshgrid*** function takes two 1D arrays and produces two 2D matrices corresponding to all pairs of (x, y) in the two arrays:

```
>>> points = np.arange(-5, 5, 0.01)    # 1000个值
>>> xs, ys = np.meshgrid(points, points)
>>> xs
array([[-5.  , -4.99, -4.98, ...,  4.97,  4.98,  4.99],
       [-5.  , -4.99, -4.98, ...,  4.97,  4.98,  4.99],
       [-5.  , -4.99, -4.98, ...,  4.97,  4.98,  4.99],
       ...,
       [-5.  , -4.99, -4.98, ...,  4.97,  4.98,  4.99],
       [-5.  , -4.99, -4.98, ...,  4.97,  4.98,  4.99],
       [-5.  , -4.99, -4.98, ...,  4.97,  4.98,  4.99]])
```

■ 输入坐标向量，返回坐标矩阵

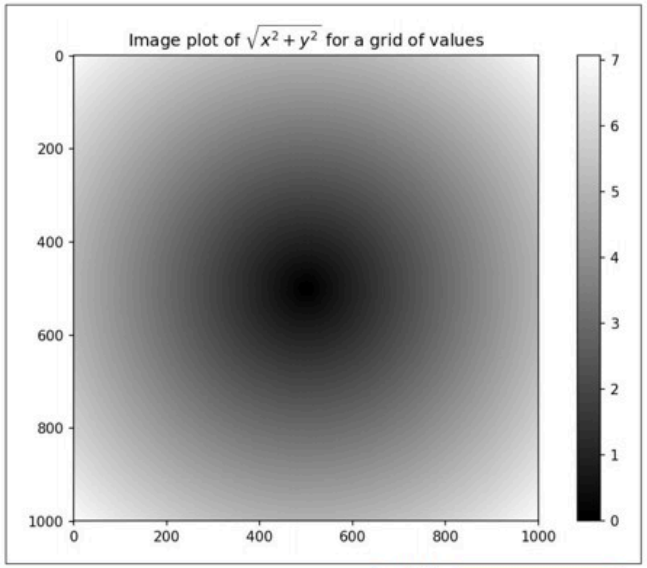
```
>>> ys
array([[-5.  , -5.  , -5.  , ..., -5.  , -5.  , -5.  ],
       [-4.99, -4.99, -4.99, ..., -4.99, -4.99, -4.99],
       [-4.98, -4.98, -4.98, ..., -4.98, -4.98, -4.98],
       ...,
       [ 4.97,  4.97,  4.97, ...,  4.97,  4.97,  4.97],
       [ 4.98,  4.98,  4.98, ...,  4.98,  4.98,  4.98],
       [ 4.99,  4.99,  4.99, ...,  4.99,  4.99,  4.99]])
```

Array-Oriented Programming with Arrays

- Example:

```
>>> z = np.sqrt(xs ** 2 + ys ** 2)
>>> z
```

```
array([[ 7.0711, 7.064 , 7.0569, ..., 7.0499, 7.0569, 7.064 ],
       [ 7.064 , 7.0569, 7.0499, ..., 7.0428, 7.0499, 7.0569],
       [ 7.0569, 7.0499, 7.0428, ..., 7.0357, 7.0428, 7.0499],
       ...,
       [ 7.0499, 7.0428, 7.0357, ..., 7.0286, 7.0357, 7.0428],
       [ 7.0569, 7.0499, 7.0428, ..., 7.0357, 7.0428, 7.0499],
       [ 7.064 , 7.0569, 7.0499, ..., 7.0428, 7.0499, 7.0569]])
```



Array-Oriented Programming with Arrays

1. Expressing Conditional Logic as Array Operations

- The numpy.where function is a vectorized version of the ternary(三元) expression:
  - *x if condition else y*

```
>>> xarr = np.array([1.1, 1.2, 1.3, 1.4, 1.5])
>>> yarr = np.array([2.1, 2.2, 2.3, 2.4, 2.5])
>>> cond = np.array([True, False, True, True, False])
>>> result = np.where(cond, xarr, yarr)
>>> result
array([ 1.1, 2.2, 1.3, 1.4, 2.5])
```

result = [(x if c else y) for x, y, c in zip(xarr, yarr, cond)]
This has multiple problems.

- First, it will not be very fast for large arrays (because all the work is being done in interpreted Python code).
- Second, it will not work with multidimensional arrays.

Array-Oriented Programming with Arrays

1. Expressing Conditional Logic as Array Operations

- The numpy.where function is a vectorized version of the ternary(三元) expression:
  - *x if condition else y*

```
>>> arr = np.random.randn(4, 4)
>>> arr
array([[-0.5031, -0.6223, -0.9212, -0.7262],
       [ 0.2229, 0.0513, -1.1577, 0.8167],
       [ 0.4336, 1.0107, 1.8249, -0.9975],
       [ 0.8506, -0.1316, 0.9124, 0.1882]])
```

```
>>> arr > 0
array([[False, False, False, False],
       [ True, True, False, True],
       [ True, True, True, False],
       [ True, False, True, True]], dtype=bool)
```

```
>>> np.where(arr>0, 2, -2)
array([[-2, -2, -2, -2],
       [ 2, 2, -2, 2],
       [ 2, 2, 2, -2],
       [ 2, -2, 2, 2]])
```

```
>>> np.where(arr < 0, 0, arr)    # 把所有<0的值设置为0
array([[ 0 ,      0 ,      0 ,      0      ],
       [ 0.2229, 0.0513,  0 ,      0.8167],
       [ 0.4336, 1.0107, 1.8249,  0      ],
       [ 0.8506, 0 ,      0.9124, 0.1882]])
```

Array-Oriented Programming with Arrays

2. Mathematical and Statistical Methods

- A set of mathematical functions that compute statistics about an entire array or about the data along an axis are accessible as methods of the array class.
  - You can use aggregations(聚合) like sum, mean, and std(标准差) either by calling the array instance method(数组实例方法) or using the top-level NumPy function(np方法).

```
>>> arr = np.random.randn(5, 4)
>>> arr
array([[ 2.1695, -0.1149, 2.0037, 0.0296],
       [ 0.7953, 0.1181, -0.7485, 0.585 ],
       [ 0.1527, -1.5657, -0.5625, -0.0327],
       [-0.929 , -0.4826, -0.0363, 1.0954],
       [ 0.9809, -0.5895, 1.5817, -0.5287]])
>>> arr.mean()          # 数组的实例方法
0.19607051119998253
>>> np.mean(arr)        # Numpy的顶层方法
0.19607051119998253
```

```
>>> arr.sum()           # 数组的实例方法
3.9214102239996507
>>> np.sum(arr)         # Numpy的顶层方法
3.9214102239996507
>>> arr.mean(axis=1)    # 某个轴向的聚合方法
array([ 1.022 , 0.1875, -0.502 , -0.0881, 0.3611])
>>> arr.sum(axis=0)
array([ 3.1693, -2.6345, 2.2381, 1.1486])
```


雨课堂 Rain Classroom

Array-Oriented Programming with Arrays

2. Mathematical and Statistical Methods

- Other methods like cumsum and cumprod do not aggregate, instead producing an array of the intermediate results:

```
>>> arr = np.arange(2,10)
>>> arr
array([2, 3, 4, 5, 6, 7, 8, 9])
>>> arr.cumsum()      # 累加函数不聚合，而是生成中间结果数组
array([ 2,  5,  9, 14, 20, 27, 35, 44], dtype=int32)
>>> arr.cumprod()     # 累积
array([    2,     6,    24,   120,   720,  5040, 40320, 362880], dtype=int32)
```


雨课堂 Rain Classroom

Array-Oriented Programming with Arrays

2. Mathematical and Statistical Methods

- In multidimensional arrays, accumulation functions like cumsum return an array of the same size, but with the partial aggregates computed along the indicated axis according to each lower dimensional slice:

```
>>> arr = np.array([[0, 1, 2], [3, 4, 5], [6, 7, 8]])
>>> arr
array([[0, 1, 2],
       [3, 4, 5],
       [6, 7, 8]])
```

```
>>> arr.cumsum(axis=0)
array([[ 0, 1, 2],
       [ 3, 5, 7],
       [ 9, 12, 15]])
```

```
>>> arr.cumprod(axis=1)
array([[ 0,   0,   0],
       [ 3,  12,  60],
       [ 6,  42, 336]])
```


雨课堂 Rain Classroom

Array-Oriented Programming with Arrays

2. Mathematical and Statistical Methods

- Basic array statistical methods:

Method	Description
sum	Sum of all the elements in the array or along an axis; zero-length arrays have sum 0
mean	Arithmetic mean; zero-length arrays have NaN mean
std, var	Standard deviation标准差 and variance方差, respectively, with optional degrees of freedom adjustment (default denominator n)
min, max	Minimum and maximum
argmin, argmax	Indices of minimum and maximum elements, respectively（最小和最大元素的索引）
cumsum	Cumulative sum of elements starting from 0
cumprod	Cumulative product of elements starting from 0

雨课堂 Rain Classroom

Array-Oriented Programming with Arrays

3. Methods for Boolean Arrays

- Boolean values are coerced(强制) to 1 (True) and 0 (False) in the preceding methods.
- Thus, ***sum*** is often used as a means of counting True values in a boolean array:

```
>>> arr = np.random.randn(100)
>>> arr>0          # 创建一个布尔数组
array([ True,  True, False,  True, False,  True, False, False, False, True,
        False,  True, False,  True, False,  True,  True,  True, True, False,
        False,  True, False, False, False,  True,  True, False, False, True,
         True, False, False, False,  True,  True,  True, False,  True, False,
        False, False,  True,  True,  True, False,  True,  True,  True, False,
        False,  True,  True, False, False, False, False,  True, False,  True,
         True,  True, False, False, False, False, False,  True, False,  True,
        False, False, False, False,  True, False, False,  True, False,  True,
         True, False, False, False, False, False,  True,  True,  True,  True,
         True,  True, False, False,  True,  True,  True,  True,  True,  True])
>>> (arr>0).sum()
50
```


雨课堂 Rain Classroom

Array-Oriented Programming with Arrays

3. Methods for Boolean Arrays

- There are two additional methods, ***any*** and ***all***, useful especially for boolean arrays.
  - ***any*** tests whether one or more values in an array is True,
  - while ***all*** checks if every value is True:

```
>>> bools = np.array([False, False, True, False])
>>> bools          # 创建一个布尔数组
array([False, False,  True, False])
>>> bools.any()      # 测试布尔数组中是否有True
True
>>> bools.all()      # 测试布尔数组中是否全都是True
False
```

- These methods also work with non-boolean arrays, where non-zero elements evaluate to True.

Array-Oriented Programming with Arrays

4. Direct Sorts

- Like Python's built-in list, the ndarray sort instance method(实例方法) is an in-place sort(就地排序), meaning that the array contents are rearranged without producing a new array:

```
>>> arr = np.random.randn(6)
>>> arr
array([ 0.82216394, -0.18174951,  0.22230836,  0.37930105, -0.78155801, 0.00287705])
>>> arr.sort()
>>> arr
array([-0.78155801, -0.18174951,  0.00287705,  0.22230836,  0.37930105, 0.82216394])
```

Array-Oriented Programming with Arrays

4. Direct Sorts

- When sorting arrays in-place, remember that if the array is a view on a different ndarray, the original array will be modified:

```
>>> arr = np.random.randn(3, 5)
>>> arr
array([[-0.3318, -1.4711, 0.8705, -0.0847, -1.1329],
       [-1.0111, -0.3436, 2.1714,  0.1234, -0.0189],
       [ 0.1773,  0.7424, 0.8548,  1.038 ,  -0.329 ]])
>>> arr[:, 0].sort()      # Sort first column values in-place
>>> arr                   # 排序后原数组改变
array([[-1.0111, -1.4711, 0.8705, -0.0847, -1.1329],
       [-0.3318, -0.3436, 2.1714,  0.1234, -0.0189],
       [ 0.1773,  0.7424, 0.8548,  1.038 ,  -0.329 ]])
```


雨课堂 Rain Classroom

Array-Oriented Programming with Arrays

4. Direct Sorts

- On the other hand, *numpy.sort* creates a new, sorted copy of an array.
- Otherwise, it accepts the same arguments (such as kind) as ndarray.sort:

```
>>> arr
array([[-1.0111, -1.4711, 0.8705, -0.0847, -1.1329],
       [-0.3318, -0.3436, 2.1714,  0.1234, -0.0189],
       [ 0.1773,  0.7424, 0.8548,  1.038 , -0.329 ]])
```

```
>>> np.sort(arr)          # 缺省列排序
array([[-1.4711, -1.1329, -1.0111, -0.0847,  0.8705],
       [-0.3436, -0.3318, -0.0189,  0.1234,  2.1714],
       [-0.329 ,  0.1773,  0.7424,  0.8548,  1.038 ]])
```

```
>>> arr          # 排序后原数组没变
array([[-1.0111, -1.4711, 0.8705, -0.0847, -1.1329],
       [-0.3318, -0.3436, 2.1714,  0.1234, -0.0189],
       [ 0.1773,  0.7424, 0.8548,  1.038 , -0.329 ]])
```

```
>>> np.sort(arr, axis=0)      # 行排序
array([[-1.0111, -1.4711,  0.8548, -0.0847, -1.1329],
       [-0.3318, -0.3436,  0.8705,  0.1234, -0.329 ],
       [ 0.1773,  0.7424,  2.1714,  1.038 , -0.0189]])
```

```
>>> arr          # 排序后原数组没变
array([[-1.0111, -1.4711, 0.8705, -0.0847, -1.1329],
       [-0.3318, -0.3436, 2.1714,  0.1234, -0.0189],
       [ 0.1773,  0.7424, 0.8548,  1.038 , -0.329 ]])
```


雨课堂 Rain Classroom

Array-Oriented Programming with Arrays

4. Direct Sorts

- None of the sort methods have an option to sort in descending order(降序排序).
- For a list, values[::-1] returns a list in reverse order(逆序). The same is true for ndarrays:

```
>>> arr
array([[-1.0111, -1.4711, 0.8705, -0.0847, -1.1329],
       [-0.3318, -0.3436, 2.1714,  0.1234, -0.0189],
       [ 0.1773,  0.7424, 0.8548,  1.038 , -0.329 ]])
>>> arr[:, ::-1]          # 逆序排序[:第一维全部，:第二维全部，:-1逆序]，缺省轴向为列
array([[-1.1329, -0.0847,  0.8705, -1.4711, -1.0111],
       [-0.0189,  0.1234,  2.1714, -0.3436, -0.3318],
       [-0.329 ,  1.038 ,  0.8548,  0.7424,  0.1773]])
```

思考题：数组如何进行降序排序?

雨课堂 Rain Classroom

Array-Oriented Programming with Arrays

5. Indirect Sorts: argsort

- In data analysis you may need to reorder datasets by one or more keys.(按key排序).

```
>>> values = np.array([5, 0, 1, 3, 2])
>>> indexer = values.argsort()        # values没有被改变，返回了一个新的数组
>>> indexer                           # 返回值为重新排序值的下标
array([1, 2, 4, 3, 0], dtype=int64)   # v1<v2<v4<v3<v0

>>> values[indexer]
array([0, 1, 2, 3, 5])
```


雨课堂 Rain Classroom

Array-Oriented Programming with Arrays

5. Indirect Sorts: argsort

- As a more complicated example, this code reorders a two-dimensional array by its first row:

```
>>> arr = np.random.randn(3, 5)
>>> arr
array([[-0.23437249,  0.34473822, -0.22791748, -0.84732268, -0.61203483],
       [ 0.83657012, -1.59456088,  0.073199 ,  1.4659142 ,  1.35870535],
       [ 1.20910467, -0.75355325, -1.13784741, -0.32957084, -1.08351376]])
>>> arr[0] = values
>>> arr
array([[ 5.        ,  0.        ,  1.        ,  3.        ,  2.        ],
       [ 0.83657012, -1.59456088,  0.073199 ,  1.4659142 ,  1.35870535],
       [ 1.20910467, -0.75355325, -1.13784741, -0.32957084, -1.08351376]])
>>> arr[:, arr[0].argsort()]          # 行不变，列按照第0行的大小顺序排序
array([[ 0.        ,  1.        ,  2.        ,  3.        ,  5.        ],
       [-1.59456088,  0.073199 ,  1.35870535,  1.4659142 ,  0.83657012],
       [-0.75355325, -1.13784741, -1.08351376, -0.32957084,  1.20910467]])
```


雨课堂 Rain Classroom

Array-Oriented Programming with Arrays

6. Unique and Other Set Logic

- NumPy has some basic set operations for one-dimensional ndarrays.
- A commonly used one is ***np.unique***, which returns the sorted unique values in an array:

```
>>> names = np.array(['Bob', 'Joe', 'Will', 'Bob', 'Will', 'Joe', 'Joe'])
>>> np.unique(names)
array(['Bob', 'Joe', 'Will'], dtype='<U4')
>>> ints = np.array([3, 3, 3, 2, 2, 1, 1, 4, 4])
>>> np.unique(ints)              # 既去重又排序
array([1, 2, 3, 4])
```

■ 等价于（纯Python代码：set去重，sorted排序）

```
>>> sorted(set(names))
['Bob', 'Joe', 'Will']
```

123

雨课堂 Rain Classroom

单选题 1分

在NumPy中，使用np.concatenate((arr1, arr2), axis=1)实现的操作属于哪种数组连接方式?

A 纵向连接

B 横向连接

C 斜向连接

D 无法连接

单选题 1分

在NumPy中，哪一个函数用于计算矩阵的逆?

A np.linalg.inv

B np.linalg.det

C np.linalg.qr

D np.linalg.dot

雨课堂 Rain Classroom

单选题 1分

在NumPy中，广播（broadcasting）机制允许不同形状的数组进行算术运算。关于广播规则，以下哪项说法是正确的?

A 只要数组的维度完全相同即可广播

B 数组的每个维度要么相等，要么其中一个为1

C 数组必须是二维才能广播

D 只有一维数组可以广播

雨课堂 Rain Classroom

单选题 1分

NumPy数组表达式替代显式循环的过程被称为什么?

A 向量化（vectorization）

B 序列化（serialization）

C 标准化（normalization）

D 泛化（generalization）

单选题 1分

在NumPy中，以下哪一个方法用于计算数组的标准差?

A std

B sum

C mean

D cumprod

雨课堂 Rain Classroom

单选题 1分

在NumPy中，使用numpy.sort对数组进行排序时，下列说法正确的是：

A numpy.sort会直接修改原数组

B numpy.sort会返回一个新的排序后的数组，原数组不变

C numpy.sort只能对一维数组排序

D numpy.sort没有kind参数

雨课堂 Rain Classroom

Practice

任务要求

- 读取iris数据集中的花萼长度数据（已保存为csv格式）
- 对其进行排序、去重
- 求出和、累积和、均值、标准差、方差、最小值、最大值

雨课堂 Rain Classroom

任务实现

1. 读取数据

In[1]:

```
iris_sepal_length = np.loadtxt("iris_sepal_length.csv", delimiter=",")    #读取文件
print('花萼长度表为：\n',iris_sepal_length)
```

Out[1]:

```
花萼长度表为：
[5.1 4.9 4.7 4.6 5.  5.4 4.6 5.  4.4 4.9 5.4 4.8 4.8 4.3 5.8 5.7 5.4 5.1
 5.7 5.1 5.4 5.1 4.6 5.1 4.8 5.  5.  5.2 5.2 4.7 4.8 5.4 5.2 5.5 4.9 5.
 5.5 4.9 4.4 5.1 5.  4.5 4.4 5.  5.1 4.8 5.1 4.6 5.3 5.  7.  6.4 6.9 5.5
 6.5 5.7 6.3 4.9 6.6 5.2 5.  5.9 6.  6.1 5.6 6.7 5.6 5.8 6.2 5.6 5.9 6.1
 6.3 6.1 6.4 6.6 6.8 6.7 6.  5.7 5.5 5.5 5.8 6.  5.4 6.  6.7 6.3 5.6 5.5
 5.5 6.1 5.8 5.  5.6 5.7 5.7 6.2 5.1 5.7 6.3 5.8 7.1 6.3 6.5 7.6 4.9 7.3
 6.7 7.2 6.5 6.4 6.8 5.7 5.8 6.4 6.5 7.7 7.7 6.  6.9 5.6 7.7 6.3 6.7 7.2
 6.2 6.1 6.4 7.2 7.4 7.9 6.4 6.3 6.1 7.7 6.3 6.4 6.  6.9 6.7 6.9 5.8 6.8
 6.7 6.7 6.3 6.5 6.2 5.9]
```


雨课堂 Rain Classroom

任务实现

2. 排序

In[2]:	**iris_sepal_length.sort()　#对数据进行排序** **print('排序后的花萼长度表为：', iris_sepal_length)**
Out[2]:	排序后的花萼长度表为：[4.3 4.4 4.4 4.4 4.5 4.6 4.6 4.6 4.6 4.7 4.7 4.8 4.8 4.8 4.8 4.8 4.9 4.9 4.9 4.9 4.9 4.9 4.9 5. 5. 5. 5. 5. 5. 5. 5. 5. 5. 5.1 5.1 5.1 5.1 5.1 5.1 5.1 5.1 5.1 5.2 5.2 5.2 5.2 5.3 5.4 5.4 5.4 5.4 5.4 5.4 5.5 5.5 5.5 5.5 5.5 5.5 5.5 5.6 5.6 5.6 5.6 5.6 5.6 5.7 5.7 5.7 5.7 5.7 5.7 5.7 5.7 5.8 5.8 5.8 5.8 5.8 5.8 5.8 5.9 5.9 5.9 6. 6. 6. 6. 6. 6. 6.1 6.1 6.1 6.1 6.1 6.1 6.2 6.2 6.2 6.2 6.3 6.3 6.3 6.3 6.3 6.3 6.3 6.3 6.3 6.4 6.4 6.4 6.4 6.4 6.4 6.4 6.5 6.5 6.5 6.5 6.5 6.6 6.6 6.7 6.7 6.7 6.7 6.7 6.7 6.7 6.7 6.8 6.8 6.8 6.9 6.9 6.9 6.9 7. 7.1 7.2 7.2 7.2 7.3 7.4 7.6 7.7 7.7 7.7 7.7 7.9]

雨课堂 Rain Classroom

任务实现

3. 去重、求和、求均值

In[3]:	**print('去重后的花萼长度表为：', np.unique(iris_sepal_length))**
Out[3]:	去重后的花萼长度表为： [4.3 4.4 4.5 4.6 4.7 4.8 4.9 5. 5.1 5.2 5.3 5.4 5.5 5.6 5.7 5.8 5.9 6. 6.1 6.2 6.3 6.4 6.5 6.6 6.7 6.8 6.9 7. 7.1 7.2 7.3 7.4 7.6 7.7 7.9]
In[4]:	**print('花萼长度表的总和为：', np.sum(iris_sepal_length)) #计算数组总和**
Out[4]:	花萼长度表的总和为： 876.5
In[5]:	**print('花萼长度表的均值为：', np.mean(iris_sepal_length)) #计算数组均值**
Out[5]:	花萼长度表的均值为： 5.843333333333334

任务实现

4. 计算所有元素的累计和

In[6]:

```
print('花萼长度表的累计和为：\n', np.cumsum(iris_sepal_length))
```

Out[6]:

```
花萼长度表的累计和为：
[  4.3   8.7  13.1  17.5  22.   26.6  31.2  35.8  40.4  45.1  49.8  54.6
  59.4  64.2  69.   73.8  78.7  83.6  88.5  93.4  98.3 103.2 108.2 113.2
 118.2 123.2 128.2 133.2 138.2 143.2 148.2 153.2 158.3 163.4 168.5 173.6
 178.7 183.8 188.9 194.  199.1 204.3 209.5 214.7 219.9 225.2 230.6 236.
 241.4 246.8 252.2 257.6 263.1 268.6 274.1 279.6 285.1 290.6 296.1 301.7
 307.3 312.9 318.5 324.1 329.7 335.4 341.1 346.8 352.5 358.2 363.9 369.6
 375.3 381.1 386.9 392.7 398.5 404.3 410.1 415.9 421.8 427.7 433.6 439.6
 445.6 451.6 457.6 463.6 469.6 475.7 481.8 487.9 494.  500.1 506.2 512.4
 518.6 524.8 531.  537.3 543.6 549.9 556.2 562.5 568.8 575.1 581.4 587.7
 594.1 600.5 606.9 613.3 619.7 626.1 632.5 639.  645.5 652.  658.5 665.
 671.6 678.2 684.9 691.6 698.3 705.  711.7 718.4 725.1 731.8 738.6 745.4
 752.2 759.1 766.  772.9 779.8 786.8 793.9 801.1 808.3 815.5 822.8 830.2
 837.8 845.5 853.2 860.9 868.6 876.5]
```

任务实现

5. 求标准差、方差、最大值、最小值

In[7]:	**print('花萼长度表的标准差为：', np.std(iris_sepal_length))**
Out[7]:	花萼长度表的标准差为： 0.8253012917851409
In[8]:	**print('花萼长度表的方差为：', np.var(iris_sepal_length)) #计算数组方差**
Out[8]:	花萼长度表的方差为： 0.6811222222222223
In[9]:	**print('花萼长度表的最大值为：', np.max(iris_sepal_length)) #计算最大值**
Out[9]:	花萼长度表的最大值为： 7.9
In[10]:	**print('花萼长度表的最小值为：', np.min(iris_sepal_length)) #计算最小值**
Out[10]:	花萼长度表的最小值为： 4.3